



Project Title: Smart Meter Anomaly Detection Dashboard

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Application (MCA)

By

Student Name: Devendra Singh Shakya

Enrollment No: CUOL909961

Under the guidance of Guide/Mentor

Name: Vasanthi Chandran

Certificate

Copy to the certificate received from Qollabb to be pasted here.

This page is intentionally kept available for the official certificate as required by the college format. The project report titled Smart Meter Anomaly Detection Dashboard has been prepared for academic submission in partial fulfillment of the requirements of the Master of Computer Application programme.

Declaration

I, Devendra Singh Shakya, hereby solemnly declare that the project report titled Smart Meter Anomaly Detection Dashboard submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Application is my original work. The project has been carried out by me during the academic year 2025-2026 under the guidance of Vasanthi Chandran.

I further declare that the work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification. All sources of information used in this report have been acknowledged, and the implementation details are based on the submitted project repository.

Place: Gurugram Date: 16/05/2026

Student Signature: Devendra Singh Shakya Enrollment No: CUOL909961

Acknowledgement

I express my sincere gratitude to my mentor Vasanthi Chandran, faculty members, department, institution, and supporting organization for their guidance and encouragement during the successful completion of this project. Their suggestions helped me refine the scope, implementation, testing approach, and final documentation.

I also thank the open-source community whose tools made the project possible. Python, scikit-learn, NumPy, pandas, FastAPI, Flask, Chart.js, joblib, and pytest provided the foundation for model training, runtime inference, dashboard visualization, and verification. Finally, I thank my family and peers for their support during development and documentation.

Abstract / Executive Summary

Smart electricity meters generate continuous readings such as voltage, current, and power. Monitoring these readings manually is difficult, especially when abnormal behavior such as theft-like low consumption, high-power faults, device malfunction, or unusual usage spikes must be detected quickly. The Smart Meter Anomaly Detection Dashboard addresses this problem by combining anomaly detection models with a real-time web dashboard.

The project trains an Isolation Forest detector for unusual individual readings and a custom LSTM detector for recent power sequence behavior. A simulator generates live readings for multiple meters and injects controlled abnormal events. A FastAPI backend receives each reading, scores it, classifies the event, calculates confidence, and stores the recent results. A Flask and Chart.js dashboard presents the live timeline, summary counters, event cards, and algorithm-level details.

The result is a complete academic prototype that demonstrates artificial intelligence inside an IoT-style monitoring workflow. The project is not limited to offline model training; it also includes runtime inference, visualization, simulated evaluation, testing, and documentation. Future work can connect the same architecture to real smart meters, persistent storage, cloud deployment, alert notifications, and confirmed field datasets.

Table of Contents

- Chapter 1: Introduction
- Chapter 2: System Study and Literature Background
- Chapter 3: Existing System and Proposed System
- Chapter 4: System Analysis
- Chapter 5: System Design
- Chapter 6: Implementation
- Chapter 7: Testing and Evaluation
- Chapter 8: Results and Discussion
- Chapter 9: Conclusion and Future Scope
- Chapter 10: References
- Appendix A: Project Viva Notes
- Appendix B: Source Code Listings

Chapter 1: Introduction

Electricity distribution has moved from periodic manual meter reading toward continuous digital monitoring. Smart meters produce frequent readings for voltage, current, and power. These values describe the electrical condition of a consumer connection and can show whether the connection is operating normally. When the number of meters grows, manual inspection becomes slow and unreliable. A monitoring system must therefore collect readings, process them automatically, and highlight suspicious behavior as early as possible.

The project titled Smart Meter Anomaly Detection Dashboard demonstrates this idea with a local software system. It trains anomaly detection models from baseline meter readings, receives live readings from a simulator, classifies each reading, and presents results on a browser dashboard. The implementation uses Python, FastAPI, Flask, Chart.js, scikit-learn, NumPy, pandas, joblib, and pytest. The work is designed as an academic project that shows how artificial intelligence can be connected with an IoT-style monitoring pipeline.

The main problem addressed by the project is the difficulty of detecting abnormal meter behavior in real time. Abnormal behavior may indicate theft-like low usage, fault-like high usage, defective devices, wiring issues, or unusual consumption. The project does not claim to replace a production utility system. Instead, it provides a complete prototype that demonstrates the data flow, model training, inference logic, dashboard visualization, and testing approach needed for such a system.

Need for smart meter monitoring

Need for smart meter monitoring is an important part of this project because meter readings arrive continuously and the operator cannot inspect every value manually with equal attention. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the simulator sending readings, the backend analyzing them, and the dashboard exposing the final decision. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats need for smart meter monitoring as a measurable concern. The evidence available inside the repository is `meter_simulator.py`, `backend.py`, `flask_app.py`, and `templates/index.html` working as a connected flow. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to slow fault discovery, missed theft-like patterns, delayed action, and weak visibility into live behavior. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is

understandable, but it also leaves a clear future direction: connecting real smart meter hardware or a cloud IoT gateway to the same backend interface. This makes need for smart meter monitoring both a completed feature of the submitted project and a foundation for further development.

Purpose of anomaly detection

Purpose of anomaly detection is an important part of this project because a smart meter system needs to find readings that differ from normal operational patterns without manually writing every possible rule. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through Isolation Forest for individual unusual readings and an LSTM detector for sequence-aware power behavior. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats purpose of anomaly detection as a measurable concern. The evidence available inside the repository is `train_model.py`, `anomaly_detection.py`, and the saved artifacts under the artifacts directory. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to fixed thresholds that fail when normal consumption changes by hour, consumer, or season. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding adaptive thresholds, model retraining schedules, and meter-wise historical profiles. This makes purpose of anomaly detection both a completed feature of the submitted project and a foundation for further development.

Academic relevance

Academic relevance is an important part of this project because the work combines programming, machine learning, web application development, testing, and documentation. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through a repository structure where each project concept has a specific source file and testable behavior. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats academic relevance as a measurable concern. The evidence available inside the repository is `README.md`, tests, artifacts, dashboard code, and this final report. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a project that appears theoretical but cannot be demonstrated or evaluated through working software. Therefore,

the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: extending the same architecture into deployment, database storage, and alert delivery channels. This makes academic relevance both a completed feature of the submitted project and a foundation for further development.

Scope of the prototype

Scope of the prototype is an important part of this project because the project must be complete enough to run locally while still being small enough to explain during evaluation. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through a local direct API mode in which the simulator posts readings to FastAPI and Flask renders dashboard data. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats scope of the prototype as a measurable concern. The evidence available inside the repository is `run_project.py` offering a one-command way to start the training and runtime components. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to scope creep, unclear deployment assumptions, and unnecessary external service dependencies. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: using the same interfaces for cloud hosting, MQTT ingestion, or a database-backed monitoring system. This makes scope of the prototype both a completed feature of the submitted project and a foundation for further development.

Objectives

- To collect and prepare smart meter readings containing voltage, current, and power.
- To train an Isolation Forest detector for point-wise anomaly detection.
- To implement an LSTM-based sequence detector for recent power behavior.
- To build a backend that receives live readings and combines model signals.
- To create a dashboard that displays recent readings, events, decisions, and confidence values.

- To simulate theft-like and fault-like events for repeatable testing.
- To provide automated tests for the most important backend and model behaviors.

Chapter 2: System Study and Literature Background

A smart meter monitoring project belongs to the combined area of IoT telemetry, time-series analysis, anomaly detection, and web visualization. The meter is the data-producing unit, the backend is the decision-making unit, and the dashboard is the human-facing unit. A successful design must keep all three areas connected. If the model is strong but the dashboard is unclear, the user cannot act on the result. If the dashboard is attractive but the model output is weak, the system cannot support trustworthy monitoring.

Traditional systems often depend on monthly reading collection, fixed limits, or manual inspection. Such methods can detect obvious errors, but they are weak against changing patterns. A household can use more power during evening hours, a business can have different weekday and weekend behavior, and a device can fail gradually. A model-based approach gives the project a better basis for learning what normal readings look like and for identifying readings that are unusual compared with that baseline.

Smart meter telemetry

Smart meter telemetry is an important part of this project because voltage, current, and power values provide a compact description of the operating state of an electrical connection. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the project using these three values as `FEATURE_NAMES` and carrying them consistently through training and inference. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats smart meter telemetry as a measurable concern. The evidence available inside the repository is `FEATURE_NAMES` in `anomaly_detection.py` and the columns used by `sample_data.csv`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to mismatched data columns, weak feature interpretation, and errors when training data differs from runtime readings. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding frequency, power factor, energy usage, location, consumer category, and tariff period features. This makes smart meter telemetry both a completed feature of the submitted project and a foundation for further development.

Time-series behavior

Time-series behavior is an important part of this project because power consumption is influenced by time and recent history, so the previous readings help interpret the next reading. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the

implementation through window creation and sequence scoring inside the custom LSTM detector. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats time-series behavior as a measurable concern. The evidence available inside the repository is `make_windows` and `LSTMANomalyDetector` in `anomaly_detection.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to models that mark a value as normal even when it is abnormal in the recent sequence context. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding longer per-device histories and meter-specific temporal profiles. This makes time-series behavior both a completed feature of the submitted project and a foundation for further development.

Point anomaly detection

Point anomaly detection is an important part of this project because some events are visible immediately because one reading is far from the normal distribution. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the Isolation Forest pipeline scoring each incoming voltage-current-power vector. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats point anomaly detection as a measurable concern. The evidence available inside the repository is `build_isolation_forest` in `train_model.py` and `isolation_forest.decision_function` in `backend.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to delayed detection of sudden high spikes, low-power theft-like events, and corrupted readings. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: combining Isolation Forest with additional models such as Local Outlier Factor or autoencoders. This makes point anomaly detection both a completed feature

of the submitted project and a foundation for further development.

Sequence anomaly detection

Sequence anomaly detection is an important part of this project because some abnormal events are clearer when recent movement is examined rather than a single value. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the LSTM detector predicting expected power from recent history and comparing the prediction error with a threshold. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats sequence anomaly detection as a measurable concern. The evidence available inside the repository is `LSTMANomalyDetector.fit`, `score`, `predict`, `save`, and `load` methods. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to false normal decisions when a single reading appears reasonable but breaks the recent pattern. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: using GRU, transformer, or rolling statistical models for deeper temporal comparison. This makes sequence anomaly detection both a completed feature of the submitted project and a foundation for further development.

Visualization for monitoring

Visualization for monitoring is an important part of this project because an anomaly alert has value only when the user can understand where it came from and how serious it appears. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the dashboard showing counters, recent event cards, algorithm status, confidence, and a power timeline. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats visualization for monitoring as a measurable concern. The evidence available inside the repository is `templates/index.html` using Chart.js, event badges, and summary cards. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to alerts that are technically generated but difficult for a user to inspect or explain. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding filters, export options, drill-down meter pages, and authenticated operator views. This makes visualization for monitoring both a completed feature of the submitted project and a foundation for further development.

Testing in AI-enabled systems

Testing in AI-enabled systems is an important part of this project because machine learning systems still require deterministic tests around data contracts, business rules, and integration behavior. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through pytest cases using fake detectors to verify backend logic without depending on random model internals. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats testing in ai-enabled systems as a measurable concern. The evidence available inside the repository is `tests/test_backend.py`, `tests/test_anomaly_detection.py`, and `tests/test_train_model.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to regressions where the dashboard, backend, or model artifacts silently disagree. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: expanding test coverage with integration tests, load tests, and stored evaluation datasets. This makes testing in ai-enabled systems both a completed feature of the submitted project and a foundation for further development.

Chapter 3: Existing System and Proposed System

Existing System

In many conventional electricity monitoring workflows, the meter reading is collected at intervals and then used for billing or auditing. The system may store the value, but it may not analyze it continuously. An operator may discover a fault after a complaint, a billing mismatch, or a physical inspection. This workflow is reactive. It does not give the organization a live view of suspicious readings.

Simple rule-based systems improve the situation by setting fixed thresholds. For example, a system may flag very high power as a fault or very low power as suspicious. However, fixed thresholds are limited because normal behavior changes across time, consumer category, and usage pattern. A threshold that works for one location may produce false alerts in another location. A threshold that works during the day may be unsuitable at night.

Proposed System

The proposed system is a real-time smart meter anomaly detection dashboard. It trains models from a baseline dataset, receives live readings from a simulator, scores every reading, classifies the event type, and presents the result visually. The system combines two approaches. Isolation Forest identifies unusual individual readings. The LSTM detector analyzes recent power sequence behavior. The backend combines these signals into a single decision for the dashboard.

The proposed system is modular. Training, anomaly detection utilities, backend inference, live simulation, dashboard serving, dashboard user interface, and tests are kept in separate files. This makes the project easier to understand and easier to demonstrate. It also helps future development because a real meter agent can replace the simulator without rewriting the model training or dashboard logic.

Requirement Summary

Requirement	Description
Input readings	Each reading contains device ID, timestamp, voltage, current, power, and simulator label.
Training	The project must train point-wise and sequence-aware anomaly detectors from baseline data.
Inference	The backend must classify every incoming reading as NORMAL or ANOMALY.
Event type	Abnormal readings must be mapped to THEFT, FAULT, or generic ANOMALY when possible.

Requirement	Description
Dashboard	Users must see live readings, power trends, event cards, confidence values, and algorithm status.
Testing	Core preprocessing, model behavior, API decisions, and dashboard-facing values must be testable.

Direct API pipeline

Direct API pipeline is an important part of this project because the prototype needs a dependable local path from generated readings to backend decisions and dashboard updates. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `meter_simulator.py` posting readings directly to the FastAPI `/data` endpoint. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats direct api pipeline as a measurable concern. The evidence available inside the repository is the `POST /data` and `GET /data` routes in `backend.py` and the Flask proxy route in `flask_app.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to loose coupling that makes the demonstration depend on manual file copying or unclear runtime steps. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: introducing MQTT, WebSockets, or cloud message queues while preserving the same API contract. This makes direct api pipeline both a completed feature of the submitted project and a foundation for further development.

Modular design

Modular design is an important part of this project because separating concerns makes the system easier to inspect, test, and extend. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through training, inference, simulation, dashboard serving, and UI code each living in its own module. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats modular design as a measurable concern. The evidence available inside the repository is the repository file structure and module responsibilities documented in README.md. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to large mixed files where model logic, web routes, and UI behavior cannot be understood independently. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: turning modules into packages and adding configuration files for multiple deployment profiles. This makes modular design both a completed feature of the submitted project and a foundation for further development.

Functional requirements

Functional requirements is an important part of this project because the project must accept readings, score them, classify event type, store recent results, and display them. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through backend functions such as `analyze_item`, `classify_event`, `combine_confidence`, and route handlers. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats functional requirements as a measurable concern. The evidence available inside the repository is tests that verify normal, theft, fault, and generic anomaly cases. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a dashboard that displays values that have not been consistently produced by the backend. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding authentication, role-based pages, and event acknowledgement workflows. This makes functional requirements both a completed feature of the submitted project and a foundation for further development.

Non-functional requirements

Non-functional requirements is an important part of this project because the application must be maintainable, understandable, testable, and suitable for a local academic demo. In the Smart Meter Anomaly

Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through clear dependencies, simple launch commands, and deterministic tests around major behaviors. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats non-functional requirements as a measurable concern. The evidence available inside the repository is requirements.txt, pytest.ini, test files, and README run instructions. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a project that works only on one machine or requires hidden manual steps. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: containerization, CI execution, and environment-specific configuration. This makes non-functional requirements both a completed feature of the submitted project and a foundation for further development.

Chapter 4: System Analysis

System analysis defines what data enters the system, how the data moves, who uses the output, and what rules must be applied. In this project the input is a stream of smart meter readings. The processing includes feature extraction, model scoring, sequence history management, confidence calculation, and event classification. The output is a set of enriched readings displayed through the dashboard.

The analysis also separates the two roles of the project. First, it is a machine learning experiment because it trains and stores anomaly detection artifacts. Second, it is a monitoring application because it receives live readings and displays decisions in real time. The report therefore treats the model and the application as parts of one monitoring loop.

Dataset Summary

Field	Observed value
Dataset path	sample_data.csv
Rows available	300
voltage	minimum 210.1617, maximum 239.9092, average 226.1608
current	minimum 5.0337, maximum 14.9998, average 10.1682
power	minimum 1.1128, maximum 3.5780, average 2.2991

Data Flow

Stage	Flow
1	sample_data.csv provides the baseline reading history.
2	train_model.py loads, prepares, and expands normal training behavior.
3	Isolation Forest and LSTM artifacts are written under artifacts/.
4	meter_simulator.py creates live meter readings and controlled spikes.
5	backend.py scores readings, combines model outputs, and returns decisions.
6	flask_app.py exposes dashboard data to the browser.

Stage	Flow
7	templates/index.html renders counters, timeline, event cards, and confidence values.

Input analysis

Input analysis is an important part of this project because the backend depends on voltage, current, and power being present in every runtime reading. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `build_feature_vector` converting input dictionaries into a fixed numeric order. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats input analysis as a measurable concern. The evidence available inside the repository is `backend.build_feature_vector` and `backend.build_feature_frame`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to wrong predictions caused by missing values, swapped columns, or inconsistent field names. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding validation models, schema definitions, and richer input error messages. This makes input analysis both a completed feature of the submitted project and a foundation for further development.

Output analysis

Output analysis is an important part of this project because the dashboard requires more than a binary model label to be useful during monitoring. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through enriched readings containing `prediction`, `detected_event`, `confidence`, `match`, `anomaly_source`, and `algorithm` details. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats output analysis as a measurable concern. The evidence available inside the repository is the `enriched_item` dictionary returned by `analyze_item` in `backend.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to unclear

alerts where the user cannot tell whether Isolation Forest, LSTM, or both produced the decision. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: creating alert tickets, downloadable reports, and long-term event histories. This makes output analysis both a completed feature of the submitted project and a foundation for further development.

Actor analysis

Actor analysis is an important part of this project because the system has multiple practical users even though it runs as a local academic prototype. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through student users running the demo, faculty evaluators inspecting design, and utility-style users reading the dashboard. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats actor analysis as a measurable concern. The evidence available inside the repository is README instructions, dashboard metrics, and report chapters describing user-facing behavior. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a system that satisfies code execution but is difficult to explain to non-developer evaluators. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding user accounts for administrator, operator, auditor, and maintenance engineer roles. This makes actor analysis both a completed feature of the submitted project and a foundation for further development.

Decision analysis

Decision analysis is an important part of this project because model output must be translated into a monitoring event that can be understood quickly. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `classify_event` mapping abnormal low power to THEFT and abnormal high power to FAULT. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user

during a live project demonstration.

From a system engineering view, the project treats decision analysis as a measurable concern. The evidence available inside the repository is the `power_low_threshold` and `power_high_threshold` metadata used in `backend.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to raw anomaly scores that are mathematically correct but operationally unclear. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: using domain-approved thresholds per consumer category and adding investigation states. This makes decision analysis both a completed feature of the submitted project and a foundation for further development.

Risk analysis

Risk analysis is an important part of this project because an anomaly detection dashboard can create false positives and false negatives if not designed carefully. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the project exposing confidence values and algorithm source instead of hiding uncertainty. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats risk analysis as a measurable concern. The evidence available inside the repository is `combine_confidence` and dashboard tooltips showing per-algorithm details. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to overconfident alerts, ignored warnings, and weak trust in the monitoring application. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding calibrated probabilities, human feedback, and retraining based on confirmed cases. This makes risk analysis both a completed feature of the submitted project and a foundation for further development.

Chapter 5: System Design

System design converts the analysis into a structure that can be implemented and tested. The design of this project follows a pipeline architecture. Baseline data is prepared, models are trained, artifacts are stored, live readings are generated, the backend performs inference, and the dashboard displays results. Each step has a clear input and output.

The design avoids unnecessary external services. This is important for an academic demonstration because the project should run reliably on a local machine. The same design can later be extended to cloud deployment, but the submitted version focuses on a direct and repeatable local workflow.

Module Design

Module	Responsibility
sample_data.csv	Historical baseline readings for voltage, current, and power.
train_model.py	Builds the Isolation Forest model, custom LSTM detector, and metadata artifacts.
anomaly_detection.py	Contains LSTM utilities, model save/load support, windows, scaler logic, and metadata helpers.
backend.py	FastAPI service that receives readings, scores them, combines signals, and stores recent decisions.
meter_simulator.py	Generates live readings for multiple meters and injects theft-like and fault-like events.
flask_app.py	Serves the dashboard page and proxies processed readings for the browser.
templates/index.html	Chart.js dashboard for metrics, timeline, recent events, and model status.
run_project.py	One-command launcher for training, backend, dashboard, and simulator processes.

Model Artifact Design

Artifact	Purpose
artifacts/isolation_forest.joblib	Serialized Isolation Forest pipeline with scaler and detector.

Artifact	Purpose
artifacts/lstm_detector.npz	Saved custom LSTM detector parameters, scaling values, and threshold.
artifacts/detector_metadata.joblib	Feature names, power thresholds, training rows, and runtime metadata.

Training design

Training design is an important part of this project because model artifacts must be produced before runtime inference can begin. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `train_model.py` loading baseline data, adding simulator-like normal rows, and saving detectors. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats training design as a measurable concern. The evidence available inside the repository is joblib files, npz files, and metadata generated under artifacts. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to runtime failures caused by missing models or incompatible training behavior. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: scheduled retraining, versioned artifacts, and model registry support. This makes training design both a completed feature of the submitted project and a foundation for further development.

Backend design

Backend design is an important part of this project because all live readings need a single decision service that can be called by simulator or future meter agents. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through FastAPI routes, CORS setup, recent DATA storage, and per-device sequence history. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats backend design as a measurable concern. The evidence available inside the repository is `backend.py` with `POST /data` and `GET /data` endpoints. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to duplicated decision logic inside multiple clients and inconsistent alert decisions. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: database persistence, authentication, async queues, and deployment behind a reverse proxy. This makes backend design both a completed feature of the submitted project and a foundation for further development.

Sequence history design

Sequence history design is an important part of this project because LSTM analysis requires recent readings to be kept separately for each meter. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `DEVICE_HISTORY` storing a bounded deque per device ID. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats sequence history design as a measurable concern. The evidence available inside the repository is tests that verify history is kept separate for meter-a and meter-b. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to one device influencing another device's LSTM context and creating incorrect anomaly scores. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: storing long-term per-meter history in a database or streaming state store. This makes sequence history design both a completed feature of the submitted project and a foundation for further development.

Dashboard design

Dashboard design is an important part of this project because monitoring output must be easy to scan during a live run. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through summary cards, event badges, a power timeline, tooltips, and live pause/resume behavior. This connection helps the report explain why the selected modules

exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats dashboard design as a measurable concern. The evidence available inside the repository is `templates/index.html` with Chart.js and dashboard JavaScript functions. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to users missing alerts or misunderstanding which model produced a warning. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding date range filters, meter selection, dark mode, and exportable incident reports. This makes dashboard design both a completed feature of the submitted project and a foundation for further development.

Testing design

Testing design is an important part of this project because important behavior should be verified without depending on random live simulation. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through fake model objects and pytest fixtures controlling backend decisions. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats testing design as a measurable concern. The evidence available inside the repository is `tests/test_backend.py` installing fake Isolation Forest and LSTM detectors. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to tests that become flaky because they wait for random spikes from the simulator. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding integration smoke tests for the complete launch flow and browser dashboard. This makes testing design both a completed feature of the submitted project and a foundation for further development.

Chapter 6: Implementation

The implementation is written mainly in Python, with HTML, CSS, and JavaScript used for the dashboard. The project does not use a database in the submitted version. Instead, it keeps a bounded list of recent processed readings in memory. This choice keeps the demonstration simple while still showing the complete inference and visualization workflow.

The training process combines the historical sample data with generated normal simulator-style rows. This helps the model understand the range of values that the simulator will produce during a live demonstration. The Isolation Forest is trained with a StandardScaler and the LSTM detector is trained on power sequences. The backend loads both artifacts when the FastAPI application starts.

Data preparation implementation

Data preparation implementation is an important part of this project because the model should learn from clean numeric values rather than raw mixed-format input. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `load_sample_frame` selecting `FEATURE_NAMES` and `generate_simulator_normal_frame` producing normal runtime patterns. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats data preparation implementation as a measurable concern. The evidence available inside the repository is `train_model.py` functions `load_sample_frame`, `generate_simulator_normal_frame`, and `load_training_frame`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to training on columns that are not available during runtime or missing normal usage variation. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding data validation reports, missing value handling, and feature engineering steps. This makes data preparation implementation both a completed feature of the submitted project and a foundation for further development.

Isolation Forest implementation

Isolation Forest implementation is an important part of this project because point-wise abnormal readings can be caught quickly with an unsupervised outlier model. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through a scikit-learn Pipeline with StandardScaler and IsolationForest using `contamination 0.04`. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the

demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats isolation forest implementation as a measurable concern. The evidence available inside the repository is `build_isolation_forest` in `train_model.py` and confidence conversion in `backend.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to manual thresholds that do not adapt to feature scale or natural variation. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: evaluating contamination settings and adding model comparison experiments. This makes isolation forest implementation both a completed feature of the submitted project and a foundation for further development.

Custom LSTM implementation

Custom LSTM implementation is an important part of this project because sequence learning helps the project demonstrate deeper AI logic beyond a single library call. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `SimpleLSTMRegressor` implementing gates, Adam optimization, save/load support, and thresholded prediction error. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats custom lstm implementation as a measurable concern. The evidence available inside the repository is `anomaly_detection.py` containing `sigmoid`, `make_windows`, `AdamOptimizer`, `SimpleLSTMRegressor`, and `LSTMANomalyDetector`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a black-box model that cannot be explained during viva or debugged from the repository. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: switching to PyTorch or TensorFlow for production-grade training after the academic concept is proven. This makes custom lstm implementation both a completed feature of the submitted project and a foundation for further development.

Runtime classification implementation

Runtime classification implementation is an important part of this project because raw model outputs need to be converted into meaningful monitoring decisions. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `backend.analyze_item` scoring features, managing LSTM warmup, combining signals, and appending history. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats runtime classification implementation as a measurable concern. The evidence available inside the repository is `prediction`, `detected_event`, `confidence`, `match`, `anomaly_source`, and algorithms in each enriched reading. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to dashboard values that are hard-coded or detached from real inference. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding severity levels, root cause categories, and operator acknowledgement fields. This makes runtime classification implementation both a completed feature of the submitted project and a foundation for further development.

Simulator implementation

Simulator implementation is an important part of this project because repeatable live readings are needed for demonstration without real meter hardware. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `meter_simulator.py` generating three device IDs, applying time-of-day effects, and injecting controlled spikes. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats simulator implementation as a measurable concern. The evidence available inside the repository is `DEVICE_IDS`, `generate_data`, and `direct requests.post` calls. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a project that can be trained but cannot show live behavior during evaluation. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is

understandable, but it also leaves a clear future direction: connecting USB, serial, MQTT, or cloud IoT readings through the same payload format. This makes simulator implementation both a completed feature of the submitted project and a foundation for further development.

Dashboard implementation

Dashboard implementation is an important part of this project because users need a visual representation of readings and anomaly decisions. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through Flask serving `index.html` and JavaScript polling `/data` every two seconds. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats dashboard implementation as a measurable concern. The evidence available inside the repository is Chart.js configuration, event card rendering, and `updateSummary/updateChart/updateEvents` functions. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to requiring users to read terminal logs to understand the model result. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: using WebSockets for push updates and adding richer analytics views. This makes dashboard implementation both a completed feature of the submitted project and a foundation for further development.

Launcher implementation

Launcher implementation is an important part of this project because a multi-process project is easier to demonstrate when startup is automated. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through `run_project.py` offering training, backend, dashboard, and simulator startup options. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats launcher implementation as a measurable concern. The evidence available inside the repository is command-line flags for debug mode and skip-training behavior. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to manual process order errors that confuse users during project evaluation. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: packaging the project with scripts, containers, or a service manager. This makes launcher implementation both a completed feature of the submitted project and a foundation for further development.

Chapter 7: Testing and Evaluation

Testing is essential because the project combines generated readings, model artifacts, API routes, and browser-facing data. A small change in the backend can affect the dashboard without producing a syntax error. The repository therefore includes pytest cases that focus on deterministic behavior. Instead of waiting for random simulator events, the tests use fake detectors and controlled inputs.

The evaluation approach is practical. The simulator attaches labels such as NORMAL, THEFT, and FAULT. The backend generates detected_event and match fields. The dashboard displays match-oriented summaries. This makes it possible to compare injected event types with detected event types during a live run, even though the project is not claiming to be a production-certified utility system.

Test Case Summary

Test area	Expected validation
Feature builders	Voltage, current, and power are converted into a correctly ordered numeric vector.
Warmup handling	LSTM remains in warmup until enough per-device history is available.
Theft classification	Low abnormal power is classified as THEFT when anomaly logic is triggered.
Fault classification	High abnormal power is classified as FAULT when anomaly logic is triggered.
Combined signals	Isolation Forest and LSTM confidence values are combined without hiding either source.
API storage	Backend keeps only the configured number of recent visible readings for the dashboard.

Unit testing strategy

Unit testing strategy is an important part of this project because core helper functions should be verified separately from web routing and random simulation. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through pytest tests checking feature builders, confidence helpers, event classification, and state restoration. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats unit testing strategy as a measurable concern. The evidence available inside the repository is `tests/test_backend.py` and `tests/test_anomaly_detection.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to hidden defects in helper logic that later appear as wrong dashboard decisions. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding property-based tests for numeric edge cases and invalid input values. This makes unit testing strategy both a completed feature of the submitted project and a foundation for further development.

Backend route testing

Backend route testing is an important part of this project because the API must store and return recent processed readings in the format expected by the dashboard. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through FastAPI TestClient posting readings and checking the recent result list. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats backend route testing as a measurable concern. The evidence available inside the repository is `test_api_routes_store_recent_results_only` in `tests/test_backend.py`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to dashboard failures caused by unexpected response shapes or unbounded memory growth. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: testing error responses, validation failures, and concurrent request behavior. This makes backend route testing both a completed feature of the submitted project and a foundation for further development.

LSTM testing

LSTM testing is an important part of this project because a sequence detector can fail silently if scaling, window creation, save, or load behavior is incorrect. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through tests checking

make_windows, detector fit/predict behavior, and persistence. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats lstm testing as a measurable concern. The evidence available inside the repository is tests/test_anomaly_detection.py covering the custom LSTM utility layer. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to model artifacts that cannot be loaded later or thresholds that are not preserved. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding comparison tests against known sequence patterns and larger evaluation datasets. This makes lstm testing both a completed feature of the submitted project and a foundation for further development.

Training pipeline testing

Training pipeline testing is an important part of this project because model training should create the required artifacts with consistent metadata. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through tests around training frame loading, simulator normal generation, artifact paths, and metadata values. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats training pipeline testing as a measurable concern. The evidence available inside the repository is tests/test_train_model.py and train_model.py. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a backend crash at startup because expected artifact files are missing. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: running training tests in CI and recording artifact version details. This makes training pipeline testing both a completed feature of the submitted project and a foundation for further development.

Evaluation limitations

Evaluation limitations is an important part of this project because simulated labels are useful but cannot represent every real-world consumer behavior pattern. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the report clearly treating the system as an academic prototype and not a certified utility deployment. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats evaluation limitations as a measurable concern. The evidence available inside the repository is README deployment direction and future scope sections. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to overstating model accuracy beyond the available dataset and simulator evidence. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: collecting real meter data, confirming anomalies with domain experts, and measuring precision and recall. This makes evaluation limitations both a completed feature of the submitted project and a foundation for further development.

Chapter 8: Results and Discussion

The completed project produces a working local dashboard for smart meter anomaly detection. When the project is launched, the simulator sends readings for multiple meters. The backend processes every reading and returns a decision. The dashboard updates the summary counters, timeline, event cards, confidence values, and algorithm status. This demonstrates that the project is not only a training script but a complete monitoring loop.

The most important result is integration. The Isolation Forest model catches many sudden abnormal readings. The LSTM detector contributes when there is enough recent history for a meter and the sequence error is strong enough. The backend exposes the source of each anomaly. This is helpful during evaluation because the user can explain whether the decision came from point anomaly logic, sequence anomaly logic, or both.

Dashboard result

Dashboard result is an important part of this project because the user-facing output must clearly show live system behavior. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through summary cards, the power timeline, recent event cards, and tooltip details updating from backend data. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats dashboard result as a measurable concern. The evidence available inside the repository is `templates/index.html` rendering visible readings, combined anomalies, theft, fault, and latest decision counters. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a project where model results exist only in terminal output and cannot be inspected easily. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding historical charts, date filters, meter comparison, and downloadable evaluation reports. This makes dashboard result both a completed feature of the submitted project and a foundation for further development.

Detection result

Detection result is an important part of this project because the project must identify abnormal low and high power events generated by the simulator. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through backend classification

of low abnormal power as THEFT and high abnormal power as FAULT. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats detection result as a measurable concern. The evidence available inside the repository is `classify_event` thresholds and tests for theft and fault behavior. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a model that flags anomalies but cannot categorize them into useful operational labels. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding more event categories such as voltage sag, over-voltage, tampering, and communication dropout. This makes detection result both a completed feature of the submitted project and a foundation for further development.

Confidence result

Confidence result is an important part of this project because monitoring users benefit from seeing how strong the alert signal appears. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the backend converting model scores into confidence values and selecting the strongest anomaly signal. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats confidence result as a measurable concern. The evidence available inside the repository is `isolation_forest_confidence`, `lstm_confidence`, and `combine_confidence`. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to binary alerts that do not communicate uncertainty and may be ignored or overtrusted. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: using calibrated probabilities and confidence explanations based on confirmed evaluation data. This makes confidence result both a completed feature of the submitted project and a foundation for further development.

Project maintainability result

Project maintainability result is an important part of this project because the repository must remain understandable after submission. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through separate modules, test files, README documentation, and a reproducible report builder. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats project maintainability result as a measurable concern. The evidence available inside the repository is the source files listed in the appendix and the repository documentation. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to a project that is hard to rebuild, explain, or extend after final submission. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is understandable, but it also leaves a clear future direction: adding continuous integration, package structure, environment files, and deployment documentation. This makes project maintainability result both a completed feature of the submitted project and a foundation for further development.

Discussion of constraints

Discussion of constraints is an important part of this project because the project is a prototype and its conclusions must remain honest about data and environment limits. In the Smart Meter Anomaly Detection Dashboard, the concept is not kept as a detached theory topic. It is connected with the implementation through the report describing simulator-based evaluation and future real meter integration. This connection helps the report explain why the selected modules exist, why the selected data fields are sufficient for the demonstration, and how the dashboard can be used by a student, faculty evaluator, or utility monitoring user during a live project demonstration.

From a system engineering view, the project treats discussion of constraints as a measurable concern. The evidence available inside the repository is `sample_data.csv`, generated normal rows, simulator labels, and future scope notes. This evidence is useful because the evaluator can move from the report to the source code and confirm that the claim has a working implementation. If this concern was ignored, the project would be exposed to unsupported claims about field accuracy or production readiness. Therefore, the analysis, design, implementation, and testing chapters all describe how this concern is managed in a practical way.

During execution, the same point can be demonstrated without special equipment. The local simulator, FastAPI backend, Flask dashboard, saved model artifacts, and pytest cases provide a complete academic demonstration environment. The current version is kept intentionally simple so that the workflow is

understandable, but it also leaves a clear future direction: evaluating the same models on real smart meter datasets and confirmed incident records. This makes discussion of constraints both a completed feature of the submitted project and a foundation for further development.

Chapter 9: Conclusion and Future Scope

The Smart Meter Anomaly Detection Dashboard successfully demonstrates a complete local monitoring workflow. It starts with baseline smart meter data, trains anomaly detection models, receives live simulated readings, scores each reading in the backend, classifies event type, and displays the result on a dashboard. The implementation shows the practical relationship between machine learning and application development.

The project also demonstrates that a useful AI system needs more than a model. It needs preprocessing, runtime data contracts, confidence handling, visualization, testing, and documentation. By separating these concerns into different modules, the project becomes easier to run and easier to explain. This is valuable for academic evaluation because the evaluator can inspect the source code and connect it with the report chapters.

The current system is intentionally limited to local execution and simulated data. This keeps the submitted project stable and demonstrable. In future work, the same design can be connected to real meters, cloud infrastructure, persistent storage, external alerts, and a richer evaluation process. The modular architecture supports these extensions without discarding the existing training, backend, and dashboard concepts.

Future Scope

- Integrate real smart meter devices using serial, MQTT, HTTP, or a cloud IoT gateway.
- Store long-term readings, model outputs, and confirmed incidents in a database.
- Add SMS, email, or push notifications for critical theft-like and fault-like alerts.
- Create authenticated dashboards for administrator, operator, and auditor roles.
- Train meter-specific models so that each meter can learn its own normal behavior.
- Add advanced evaluation metrics such as precision, recall, F1 score, false positive rate, and detection delay.
- Deploy the backend and dashboard through Docker, cloud hosting, or a campus server.
- Add explainability panels that show why a reading was marked as abnormal.

Chapter 10: References

The project uses standard open-source tools and common machine learning concepts. The following references are included for academic completeness. They describe the libraries, frameworks, and general model families used in the implementation.

- scikit-learn documentation: IsolationForest and preprocessing pipeline concepts.
- NumPy documentation: array operations, numerical processing, and npz persistence.
- pandas documentation: CSV loading and dataframe operations.
- FastAPI documentation: API route handling and request-response behavior.
- Flask documentation: lightweight web application routing and template rendering.
- Chart.js documentation: browser-based chart rendering and tooltips.
- pytest documentation: automated testing, fixtures, and assertions.
- General study material on time-series anomaly detection, smart meter analytics, and IoT monitoring architecture.

Appendix A: Project Viva Notes

Why are both Isolation Forest and LSTM used?

Isolation Forest detects unusual individual readings, while the LSTM detector checks whether the newest power value is unusual compared with recent sequence history. Using both makes the prototype easier to explain because it shows point anomaly detection and sequence anomaly detection in one workflow.

What does the backend return for each reading?

The backend returns an enriched reading with prediction, detected_event, confidence, match, anomaly_source, and per-algorithm details. These fields are used directly by the dashboard cards, event list, and chart tooltips.

How does the simulator help the project?

The simulator replaces real hardware for the academic demo. It produces normal readings for multiple meters and occasionally injects low-power theft-like events or high-power fault-like events. This gives repeatable live data for testing and presentation.

What are the limitations of the current version?

The current version uses simulated live readings and a small local dataset. It does not include persistent storage, real utility integration, authentication, or certified field evaluation. Those items are listed as future scope.

Why is testing included in the repository?

Testing verifies the helper logic, model persistence, backend decision rules, per-device history behavior, and API storage behavior. This reduces the risk that a later code change silently breaks the dashboard output.

Appendix B: Source Code Listings

The following listings are included so that the report remains directly connected with the submitted implementation. They also support the final report requirement by documenting the actual training, backend, simulator, dashboard, launcher, and test logic used in the project.

Listing: README.md

```
0001: # Smart Meter Anomaly Detection Dashboard
0002:
0003: This project is a smart meter monitoring and anomaly-detection system. It collects
    historical meter readings, preprocesses the time-series data, applies `IsolationForest`
    and `LSTM` anomaly detection, streams live readings through a direct API pipeline, and
    visualizes results on a dashboard.
0004:
0005: The system is designed as an academic demo of how AI can be integrated into smart energy
    monitoring for theft-like and fault-like behavior detection.
0006:
0007: ## Project Goals
0008:
0009: This project is built around the following goals:
```

```

0010:
0011: - collect historical smart meter readings
0012: - preprocess time-series datasets
0013: - implement anomaly detection using `IsolationForest` and `LSTM`
0014: - deploy monitoring agents that send live readings
0015: - visualize anomalies on dashboards
0016: - configure automated alert triggers
0017: - test detection behavior using simulated spikes
0018: - evaluate false positives and precision-related behavior
0019: - support deployment through a backend + dashboard architecture
0020: - document the AI workflow and monitoring integration
0021:
0022: ## Current Repository Scope
0023:
0024: The repository currently implements the full local demo version of the project:
0025:
0026: - historical baseline data in `sample_data.csv`
0027: - anomaly model training in [train_model.py] (/d:/SmartIotMCAPProject/train_model.py:1)
0028: - custom LSTM utilities in
    [anomaly_detection.py] (/d:/SmartIotMCAPProject/anomaly_detection.py:1)
0029: - FastAPI inference service in [backend.py] (/d:/SmartIotMCAPProject/backend.py:1)
0030: - live meter simulator in
    [meter_simulator.py] (/d:/SmartIotMCAPProject/meter_simulator.py:1)
0031: - Flask dashboard in [flask_app.py] (/d:/SmartIotMCAPProject/flask_app.py:1)
0032: - dashboard UI in [templates/index.html] (/d:/SmartIotMCAPProject/templates/index.html:1)
0033: - one-command launcher in [run_project.py] (/d:/SmartIotMCAPProject/run_project.py:1)
0034:
0035: ## Project Architecture
0036:
0037: ```text
0038: sample_data.csv
0039: |
0040: v
0041: train_model.py
0042: |
0043: v
0044: artifacts/
0045: |
0046: v
0047: meter_simulator.py --> backend.py --> flask_app.py --> templates/index.html
0048: ```
0049:
0050: ### Flow Summary
0051:
0052: 1. historical smart meter readings are prepared in `sample_data.csv`
0053: 2. `train_model.py` preprocesses the data and trains anomaly detectors
0054: 3. model artifacts are saved under `artifacts/`
0055: 4. `meter_simulator.py` generates live readings for multiple meters
0056: 5. `backend.py` scores each reading with `IsolationForest` and `LSTM`
0057: 6. `flask_app.py` serves dashboard data
0058: 7. the dashboard visualizes anomalies, events, and model outputs
0059:
0060: ## Historical Data Collection
0061:
0062: The historical dataset is stored in `sample_data.csv`.
0063:
0064: It provides the normal baseline behavior used for model training. The dataset contains
    the following numerical features:
0065:
0066: - `voltage`
0067: - `current`
0068: - `power`
0069:
0070: These readings represent typical smart meter operation and are used as the reference
    profile for anomaly detection.
0071:
0072: ## Time-Series Preprocessing
0073:
0074: The preprocessing workflow is handled in `train_model.py`.
0075:
0076: It includes:
0077:
0078: - selecting the three core features
0079: - combining historical and simulator-like normal patterns
0080: - preparing a clean baseline for anomaly learning
0081: - scaling features for `IsolationForest`
0082: - building fixed-length windows from the `power` series for `LSTM`
0083: - learning an anomaly threshold from normal sequence prediction error

```

```

0084:
0085: This gives the project both point-wise and sequential context during runtime.
0086:
0087: ## Anomaly Detection Models
0088:
0089: The project uses two complementary algorithms:
0090:
0091: ### `IsolationForest`
0092:
0093: - detects unusual individual readings
0094: - works well for outlier-style anomalies
0095: - helps catch sudden abnormal values immediately
0096:
0097: ### `LSTM`
0098:
0099: - analyzes recent sequential behavior
0100: - predicts the next expected `power` value
0101: - flags a sequence anomaly when prediction error becomes too high
0102:
0103: ## Runtime Decision Logic
0104:
0105: The backend combines both model signals into:
0106:
0107: - `prediction`: `NORMAL` or `ANOMALY`
0108: - `detected_event`: `NORMAL`, `THEFT`, `FAULT`, or `ANOMALY`
0109: - `confidence`
0110: - `anomaly_source`
0111:
0112: In this project:
0113:
0114: - very low abnormal power is interpreted as `THEFT`
0115: - very high abnormal power is interpreted as `FAULT`
0116:
0117: This makes the dashboard easier to explain during demos and viva presentations.
0118:
0119: ## Monitoring Agent
0120:
0121: `meter_simulator.py` acts as the live monitoring agent in this repository.
0122:
0123: It:
0124:
0125: - generates readings for multiple meter devices
0126: - stamps each reading with device ID and timestamp
0127: - simulates normal usage variation
0128: - injects abnormal low and high spikes
0129: - sends data directly to the FastAPI backend
0130:
0131: This mirrors how a real smart meter agent could forward live readings from distributed
    devices to a central monitoring service.
0132:
0133: ## Dashboard And Visualization
0134:
0135: The dashboard is served through Flask and rendered with Chart.js.
0136:
0137: It shows:
0138:
0139: - total visible readings
0140: - combined anomalies
0141: - detected theft-like events
0142: - detected fault-like events
0143: - `IsolationForest` alerts
0144: - `LSTM` alerts
0145: - latest decision
0146: - live power timeline
0147: - recent event cards
0148: - confidence values
0149: - algorithm status and anomaly source
0150:
0151: This helps users visually inspect how the detection pipeline behaves in real time.
0152:
0153: ## Automated Alert Triggers
0154:
0155: The project includes alert-driving logic inside the backend.
0156:
0157: An alert condition is triggered when:
0158:
0159: - `IsolationForest` marks the current reading as anomalous, or
0160: - `LSTM` reports a strong enough sequence anomaly to support an alert

```

```

0161:
0162: The alert state is exposed through:
0163:
0164: - `prediction`
0165: - `detected_event`
0166: - `confidence`
0167: - `anomaly_source`
0168:
0169: In the current repository, alerting is visible in the dashboard and API response. In a
    future deployed version, the same alert state can be forwarded to external channels such
    as SMS, email, or utility monitoring systems.
0170:
0171: ## Simulated Spike Testing
0172:
0173: The simulator injects controlled abnormal values to test the detection behavior.
0174:
0175: Current abnormal test cases include:
0176:
0177: - low power spikes representing theft-like behavior
0178: - high power spikes representing fault-like behavior
0179:
0180: Because the simulator also attaches a label, the backend output can be compared against
    the expected event type during testing.
0181:
0182: ## Evaluation Approach
0183:
0184: The project supports practical evaluation through simulator labels and backend
    decisions.
0185:
0186: Important evaluation indicators include:
0187:
0188: - `match count`
0189: - false positives
0190: - anomaly frequency on normal behavior
0191: - precision-style comparison between injected events and detected events
0192:
0193: The backend already returns a `match` field, and the dashboard displays match-oriented
    summaries. This makes it easier to assess whether the detectors are behaving sensibly
    during live runs.
0194:
0195: ## Model Artifacts
0196:
0197: Training saves the following artifacts under `artifacts/`:
0198:
0199: - `isolation_forest.joblib`
0200: - `lstm_detector.npz`
0201: - `detector_metadata.joblib`
0202:
0203: Generate them with:
0204:
0205: ```bash
0206: python train_model.py
0207: ```
0208:
0209: ## Installation
0210:
0211: Install dependencies with:
0212:
0213: ```bash
0214: pip install -r requirements.txt
0215: ```
0216:
0217: Main dependencies:
0218:
0219: - `pandas`
0220: - `numpy`
0221: - `scikit-learn`
0222: - `joblib`
0223: - `fastapi`
0224: - `uvicorn`
0225: - `flask`
0226: - `requests`
0227:
0228: ## How To Run
0229:
0230: ### Recommended
0231:
0232: ```bash

```

```
0233: python run_project.py
0234: ```
0235:
0236: ### Debug mode
0237:
0238: ```bash
0239: python run_project.py --debug true
0240: ```
0241:
0242: ### Reuse existing model artifacts
0243:
0244: ```bash
0245: python run_project.py --skip-training true
0246: ```
0247:
0248: ### Manual startup order
0249:
0250: 1. Train the models
0251:
0252: ```bash
0253: python train_model.py
0254: ```
0255:
0256: 2. Start the backend
0257:
0258: ```bash
0259: python -m uvicorn backend:app --host 0.0.0.0 --port 8000
0260: ```
0261:
0262: 3. Start the dashboard
0263:
0264: ```bash
0265: python flask_app.py
0266: ```
0267:
0268: 4. Start the live simulator
0269:
0270: ```bash
0271: python meter_simulator.py
0272: ```
0273:
0274: Then open:
0275:
0276: ```text
0277: http://127.0.0.1:5000
0278: ```
0279:
0280: ## API Behavior
0281:
0282: ### `POST /data`
0283:
0284: Receives a smart meter reading and returns a processed anomaly decision.
0285:
0286: ### `GET /data`
0287:
0288: Returns recent processed readings for the dashboard.
0289:
0290: Each enriched reading can include:
0291:
0292: - device data
0293: - simulator label
0294: - `prediction`
0295: - `detected_event`
0296: - `confidence`
0297: - `anomaly_source`
0298: - per-algorithm details for `IsolationForest` and `LSTM`
0299:
0300: ## Deployment Direction
0301:
0302: The current repository is a local academic demo, but its structure supports
    straightforward deployment:
0303:
0304: - host the FastAPI backend on a server
0305: - host the dashboard service with the backend or behind a reverse proxy
0306: - deploy monitoring agents where smart meter data is produced
0307: - forward alert states to external notification systems
0308: - optionally store long-term readings in a database
0309:
```

```

0310: ## AI Workflow And Monitoring Integration
0311:
0312: The AI workflow in this project is:
0313:
0314: 1. collect historical smart meter data
0315: 2. preprocess the time-series baseline
0316: 3. train anomaly detection models
0317: 4. receive live readings from monitoring agents
0318: 5. run inference continuously in the backend
0319: 6. visualize events in the dashboard
0320: 7. trigger alerts for suspicious behavior
0321: 8. evaluate false positives and match quality
0322: 9. retrain and tune thresholds if needed
0323:
0324: This is the main integration idea of the project: the AI models are part of the full
      monitoring loop, not just offline experiments.

```

Listing: train_model.py

```

0001: from __future__ import annotations
0002:
0003: import joblib
0004: import numpy as np
0005: import pandas as pd
0006: from sklearn.ensemble import IsolationForest
0007: from sklearn.pipeline import Pipeline
0008: from sklearn.preprocessing import StandardScaler
0009:
0010: from anomaly_detection import (
0011:     FEATURE_NAMES,
0012:     ISOLATION_FOREST_PATH,
0013:     LSTM_MODEL_PATH,
0014:     LSTMAnomalyDetector,
0015:     ensure_artifact_dir,
0016:     save_metadata,
0017: )
0018:
0019:
0020: RANDOM_STATE = 42
0021: ISOLATION_FOREST_CONTAMINATION = 0.04
0022: SIMULATED_NORMAL_ROWS = 480
0023:
0024:
0025: def load_sample_frame() -> pd.DataFrame:
0026:     frame = pd.read_csv("sample_data.csv")
0027:     return frame[list(FEATURE_NAMES)].copy()
0028:
0029:
0030: def generate_simulator_normal_frame(rows: int = SIMULATED_NORMAL_ROWS) -> pd.DataFrame:
0031:     rng = np.random.default_rng(RANDOM_STATE)
0032:     samples = []
0033:     for index in range(rows):
0034:         hour = index % 24
0035:         voltage = rng.uniform(210.0, 240.0)
0036:         current = rng.uniform(5.0, 15.0)
0037:         power = voltage * current / 1000.0
0038:         if 18 <= hour <= 23:
0039:             power *= 1.5
0040:         elif 0 <= hour <= 6:
0041:             power *= 0.55
0042:         power += rng.normal(0.0, 0.08)
0043:         power = max(0.6, power)
0044:         samples.append(
0045:             {
0046:                 "voltage": round(float(voltage), 4),
0047:                 "current": round(float(current), 4),
0048:                 "power": round(float(power), 4),
0049:             }
0050:         )
0051:     return pd.DataFrame(samples, columns=FEATURE_NAMES)
0052:
0053:
0054: def load_training_frame() -> tuple[pd.DataFrame, pd.DataFrame]:
0055:     sample_frame = load_sample_frame()
0056:     simulator_frame = generate_simulator_normal_frame()
0057:     combined_frame = pd.concat([sample_frame, simulator_frame], ignore_index=True)

```

```

0058:     return combined_frame, simulator_frame
0059:
0060:
0061: def build_isolation_forest(values: pd.DataFrame) -> Pipeline:
0062:     detector = Pipeline(
0063:         steps=[
0064:             ("scaler", StandardScaler()),
0065:             (
0066:                 "isolation_forest",
0067:                 IsolationForest(
0068:                     contamination=ISOLATION_FOREST_CONTAMINATION,
0069:                     n_estimators=250,
0070:                     random_state=RANDOM_STATE,
0071:                 ),
0072:             ),
0073:         ]
0074:     )
0075:     detector.fit(values)
0076:     return detector
0077:
0078:
0079: def build_lstm_detector(values: pd.DataFrame) -> LSTMAnomalyDetector:
0080:     detector = LSTMAnomalyDetector(
0081:         feature_count=1,
0082:         sequence_length=8,
0083:         hidden_size=18,
0084:         epochs=60,
0085:         learning_rate=0.003,
0086:         threshold_quantile=99.0,
0087:         random_state=RANDOM_STATE,
0088:     )
0089:     detector.fit(values[["power"]].to_numpy(dtype=float))
0090:     return detector
0091:
0092:
0093: def build_metadata(values: pd.DataFrame, lstm_detector: LSTMAnomalyDetector) ->
dict[str, float | int | str | tuple[str, ...]]:
0094:     power_min = float(values["power"].min())
0095:     power_max = float(values["power"].max())
0096:     power_std = float(values["power"].std())
0097:
0098:     return {
0099:         "features": FEATURE_NAMES,
0100:         "power_low_threshold": max(0.1, power_min - (0.35 * power_std)),
0101:         "power_high_threshold": power_max + (0.75 * power_std),
0102:         "lstm_sequence_length": lstm_detector.sequence_length,
0103:         "lstm_threshold": float(lstm_detector.threshold_ or 0.0),
0104:         "lstm_feature_name": "power",
0105:         "isolation_forest_contamination": ISOLATION_FOREST_CONTAMINATION,
0106:         "training_rows": int(len(values)),
0107:         "primary_runtime_mode": "direct",
0108:     }
0109:
0110:
0111: def main() -> None:
0112:     ensure_artifact_dir()
0113:     training_frame, lstm_training_frame = load_training_frame()
0114:     isolation_forest = build_isolation_forest(training_frame)
0115:     lstm_detector = build_lstm_detector(lstm_training_frame)
0116:     metadata = build_metadata(training_frame, lstm_detector)
0117:
0118:     joblib.dump(isolation_forest, ISOLATION_FOREST_PATH)
0119:     lstm_detector.save(LSTM_MODEL_PATH)
0120:     save_metadata(metadata)
0121:
0122:     print("Isolation Forest and LSTM anomaly detectors trained successfully.",
flush=True)
0123:     print(f"Saved Isolation Forest artifact to {ISOLATION_FOREST_PATH}", flush=True)
0124:     print(f"Saved LSTM artifact to {LSTM_MODEL_PATH}", flush=True)
0125:
0126:
0127: if __name__ == "__main__":
0128:     main()

```

Listing: anomaly_detection.py

```

0001: from __future__ import annotations
0002:
0003: from dataclasses import dataclass
0004: from pathlib import Path
0005:
0006: import joblib
0007: import numpy as np
0008:
0009:
0010: FEATURE_NAMES = ("voltage", "current", "power")
0011: ARTIFACT_DIR = Path(__file__).resolve().parent / "artifacts"
0012: ISOLATION_FOREST_PATH = ARTIFACT_DIR / "isolation_forest.joblib"
0013: LSTM_MODEL_PATH = ARTIFACT_DIR / "lstm_detector.npz"
0014: METADATA_PATH = ARTIFACT_DIR / "detector_metadata.joblib"
0015:
0016:
0017: def ensure_artifact_dir() -> None:
0018:     ARTIFACT_DIR.mkdir(exist_ok=True)
0019:
0020:
0021: def sigmoid(value: np.ndarray) -> np.ndarray:
0022:     return 1.0 / (1.0 + np.exp(-value))
0023:
0024:
0025: def make_windows(data: np.ndarray, sequence_length: int) -> tuple[np.ndarray,
    np.ndarray]:
0026:     sequences = []
0027:     targets = []
0028:     for index in range(len(data) - sequence_length):
0029:         sequences.append(data[index : index + sequence_length])
0030:         targets.append(data[index + sequence_length])
0031:     return np.asarray(sequences), np.asarray(targets)
0032:
0033:
0034: class AdamOptimizer:
0035:     def __init__(self, parameters: dict[str, np.ndarray], learning_rate: float) -> None:
0036:         self.learning_rate = learning_rate
0037:         self.beta1 = 0.9
0038:         self.beta2 = 0.999
0039:         self.epsilon = 1e-8
0040:         self.timestep = 0
0041:         self.moment_1 = {name: np.zeros_like(value) for name, value in
    parameters.items()}
0042:         self.moment_2 = {name: np.zeros_like(value) for name, value in
    parameters.items()}
0043:
0044:     def step(self, parameters: dict[str, np.ndarray], gradients: dict[str, np.ndarray])
    -> None:
0045:         self.timestep += 1
0046:         for name, gradient in gradients.items():
0047:             self.moment_1[name] = self.beta1 * self.moment_1[name] + (1.0 - self.beta1)
    * gradient
0048:             self.moment_2[name] = self.beta2 * self.moment_2[name] + (1.0 - self.beta2)
    * (gradient**2)
0049:
0050:             moment_1_hat = self.moment_1[name] / (1.0 - self.beta1**self.timestep)
0051:             moment_2_hat = self.moment_2[name] / (1.0 - self.beta2**self.timestep)
0052:             parameters[name] -= self.learning_rate * moment_1_hat /
    (np.sqrt(moment_2_hat) + self.epsilon)
0053:
0054:
0055: class SimpleLSTMRegressor:
0056:     def __init__(self, input_size: int, hidden_size: int = 16, random_state: int = 42)
    -> None:
0057:         self.input_size = input_size
0058:         self.hidden_size = hidden_size
0059:         rng = np.random.default_rng(random_state)
0060:         weight_scale = np.sqrt(2.0 / (input_size + hidden_size))
0061:
0062:         self.parameters = {
0063:             "Wf": rng.normal(0.0, weight_scale, size=(input_size, hidden_size)),
0064:             "Uf": rng.normal(0.0, weight_scale, size=(hidden_size, hidden_size)),
0065:             "bf": np.ones(hidden_size),
0066:             "Wi": rng.normal(0.0, weight_scale, size=(input_size, hidden_size)),
0067:             "Ui": rng.normal(0.0, weight_scale, size=(hidden_size, hidden_size)),
0068:             "bi": np.zeros(hidden_size),
0069:             "Wg": rng.normal(0.0, weight_scale, size=(input_size, hidden_size)),
0070:             "Ug": rng.normal(0.0, weight_scale, size=(hidden_size, hidden_size)),

```

```

0071:         "bg": np.zeros(hidden_size),
0072:         "Wo": rng.normal(0.0, weight_scale, size=(input_size, hidden_size)),
0073:         "Uo": rng.normal(0.0, weight_scale, size=(hidden_size, hidden_size)),
0074:         "bo": np.zeros(hidden_size),
0075:         "Wy": rng.normal(0.0, weight_scale, size=(hidden_size, input_size)),
0076:         "by": np.zeros(input_size),
0077:     }
0078:
0079:     def forward(self, sequence: np.ndarray) -> tuple[np.ndarray, list[dict[str,
np.ndarray]]]:
0080:         hidden_state = np.zeros(self.hidden_size)
0081:         cell_state = np.zeros(self.hidden_size)
0082:         cache: list[dict[str, np.ndarray]] = []
0083:
0084:         for x_t in sequence:
0085:             forget_gate = sigmoid(
0086:                 x_t @ self.parameters["Wf"] + hidden_state @ self.parameters["Uf"] +
self.parameters["bf"]
0087:             )
0088:             input_gate = sigmoid(
0089:                 x_t @ self.parameters["Wi"] + hidden_state @ self.parameters["Ui"] +
self.parameters["bi"]
0090:             )
0091:             candidate = np.tanh(
0092:                 x_t @ self.parameters["Wg"] + hidden_state @ self.parameters["Ug"] +
self.parameters["bg"]
0093:             )
0094:             output_gate = sigmoid(
0095:                 x_t @ self.parameters["Wo"] + hidden_state @ self.parameters["Uo"] +
self.parameters["bo"]
0096:             )
0097:
0098:             previous_hidden = hidden_state
0099:             previous_cell = cell_state
0100:             cell_state = forget_gate * cell_state + input_gate * candidate
0101:             hidden_state = output_gate * np.tanh(cell_state)
0102:
0103:             cache.append(
0104:                 {
0105:                     "x": x_t,
0106:                     "h_prev": previous_hidden,
0107:                     "c_prev": previous_cell,
0108:                     "f": forget_gate,
0109:                     "i": input_gate,
0110:                     "g": candidate,
0111:                     "o": output_gate,
0112:                     "c": cell_state,
0113:                     "h": hidden_state,
0114:                 }
0115:             )
0116:
0117:             prediction = hidden_state @ self.parameters["Wy"] + self.parameters["by"]
0118:             return prediction, cache
0119:
0120:     def backward(
0121:         self, target: np.ndarray, prediction: np.ndarray, cache: list[dict[str,
np.ndarray]]
0122:     ) -> dict[str, np.ndarray]:
0123:         gradients = {name: np.zeros_like(value) for name, value in
self.parameters.items()}
0124:         prediction_gradient = 2.0 * (prediction - target) / target.size
0125:
0126:         gradients["Wy"] += np.outer(cache[-1]["h"], prediction_gradient)
0127:         gradients["by"] += prediction_gradient
0128:
0129:         hidden_gradient = prediction_gradient @ self.parameters["Wy"].T
0130:         cell_gradient = np.zeros(self.hidden_size)
0131:
0132:         for step in reversed(cache):
0133:             tanh_cell = np.tanh(step["c"])
0134:             output_gradient = hidden_gradient * tanh_cell
0135:             output_pre_activation = output_gradient * step["o"] * (1.0 - step["o"])
0136:
0137:             cell_gradient = cell_gradient + hidden_gradient * step["o"] * (1.0 -
tanh_cell**2)
0138:             forget_gradient = cell_gradient * step["c_prev"]
0139:             forget_pre_activation = forget_gradient * step["f"] * (1.0 - step["f"])
0140:

```

```

0141:         input_gradient = cell_gradient * step["g"]
0142:         input_pre_activation = input_gradient * step["i"] * (1.0 - step["i"])
0143:
0144:         candidate_gradient = cell_gradient * step["i"]
0145:         candidate_pre_activation = candidate_gradient * (1.0 - step["g"] ** 2)
0146:
0147:         gradients["Wf"] += np.outer(step["x"], forget_pre_activation)
0148:         gradients["Uf"] += np.outer(step["h_prev"], forget_pre_activation)
0149:         gradients["bf"] += forget_pre_activation
0150:
0151:         gradients["Wi"] += np.outer(step["x"], input_pre_activation)
0152:         gradients["Ui"] += np.outer(step["h_prev"], input_pre_activation)
0153:         gradients["bi"] += input_pre_activation
0154:
0155:         gradients["Wg"] += np.outer(step["x"], candidate_pre_activation)
0156:         gradients["Ug"] += np.outer(step["h_prev"], candidate_pre_activation)
0157:         gradients["bg"] += candidate_pre_activation
0158:
0159:         gradients["Wo"] += np.outer(step["x"], output_pre_activation)
0160:         gradients["Uo"] += np.outer(step["h_prev"], output_pre_activation)
0161:         gradients["bo"] += output_pre_activation
0162:
0163:         hidden_gradient = (
0164:             forget_pre_activation @ self.parameters["Uf"].T
0165:             + input_pre_activation @ self.parameters["Ui"].T
0166:             + candidate_pre_activation @ self.parameters["Ug"].T
0167:             + output_pre_activation @ self.parameters["Uo"].T
0168:         )
0169:         cell_gradient = cell_gradient * step["f"]
0170:
0171:         for gradient in gradients.values():
0172:             np.clip(gradient, -1.0, 1.0, out=gradient)
0173:         return gradients
0174:
0175:     def fit(
0176:         self,
0177:         sequences: np.ndarray,
0178:         targets: np.ndarray,
0179:         epochs: int = 80,
0180:         learning_rate: float = 0.003,
0181:     ) -> list[float]:
0182:         optimizer = AdamOptimizer(self.parameters, learning_rate=learning_rate)
0183:         rng = np.random.default_rng(42)
0184:         losses: list[float] = []
0185:
0186:         for _ in range(epochs):
0187:             epoch_losses = []
0188:             for index in rng.permutation(len(sequences)):
0189:                 prediction, cache = self.forward(sequences[index])
0190:                 loss = float(np.mean((prediction - targets[index]) ** 2))
0191:                 gradients = self.backward(targets[index], prediction, cache)
0192:                 optimizer.step(self.parameters, gradients)
0193:                 epoch_losses.append(loss)
0194:             losses.append(float(np.mean(epoch_losses)))
0195:         return losses
0196:
0197:     def predict(self, sequence: np.ndarray) -> np.ndarray:
0198:         prediction, _ = self.forward(sequence)
0199:         return prediction
0200:
0201:
0202: @dataclass
0203: class LSTMAnomalyDetector:
0204:     feature_count: int = 1
0205:     sequence_length: int = 8
0206:     hidden_size: int = 16
0207:     epochs: int = 80
0208:     learning_rate: float = 0.003
0209:     threshold_quantile: float = 97.5
0210:     random_state: int = 42
0211:
0212:     def __post_init__(self) -> None:
0213:         self.model = SimpleLSTMRegressor(
0214:             input_size=self.feature_count,
0215:             hidden_size=self.hidden_size,
0216:             random_state=self.random_state,
0217:         )
0218:         self.feature_mean_: np.ndarray | None = None

```

```

0219:         self.feature_std_: np.ndarray | None = None
0220:         self.threshold_: float | None = None
0221:         self.training_loss_: list[float] = []
0222:
0223:     def fit(self, values: np.ndarray) -> "LSTMAnomalyDetector":
0224:         values = np.asarray(values, dtype=float)
0225:         if values.ndim == 1:
0226:             values = values.reshape(-1, 1)
0227:         if len(values) <= self.sequence_length:
0228:             raise ValueError("Not enough samples to train the LSTM detector.")
0229:
0230:         self.feature_mean_ = values.mean(axis=0)
0231:         self.feature_std_ = values.std(axis=0)
0232:         self.feature_std_[self.feature_std_ == 0.0] = 1.0
0233:
0234:         scaled = self._scale(values)
0235:         sequences, targets = make_windows(scaled, self.sequence_length)
0236:         self.training_loss_ = self.model.fit(
0237:             sequences,
0238:             targets,
0239:             epochs=self.epochs,
0240:             learning_rate=self.learning_rate,
0241:         )
0242:
0243:         errors = np.asarray([self._prediction_error(sequence, target) for sequence,
target in zip(sequences, targets)])
0244:         self.threshold_ = float(np.percentile(errors, self.threshold_quantile))
0245:         return self
0246:
0247:     def _scale(self, values: np.ndarray) -> np.ndarray:
0248:         if self.feature_mean_ is None or self.feature_std_ is None:
0249:             raise ValueError("Detector is not trained.")
0250:         values = np.asarray(values, dtype=float)
0251:         if values.ndim == 1 and self.feature_count == 1:
0252:             values = values.reshape(-1, 1) if values.size > 1 else values.reshape(1)
0253:         return (values - self.feature_mean_) / self.feature_std_
0254:
0255:     def _prediction_error(self, sequence: np.ndarray, target: np.ndarray) -> float:
0256:         predicted = self.model.predict(sequence)
0257:         return float(np.mean((predicted - target) ** 2))
0258:
0259:     def score(self, sequence: np.ndarray, target: np.ndarray) -> float:
0260:         scaled_sequence = self._scale(sequence)
0261:         scaled_target = self._scale(target)
0262:         if self.feature_count == 1:
0263:             scaled_sequence = np.asarray(scaled_sequence,
dtype=float).reshape(self.sequence_length, 1)
0264:             scaled_target = np.asarray(scaled_target, dtype=float).reshape(1)
0265:             return self._prediction_error(scaled_sequence, scaled_target)
0266:
0267:     def predict(self, sequence: np.ndarray, target: np.ndarray) -> tuple[bool, float]:
0268:         if self.threshold_ is None:
0269:             raise ValueError("Detector is not trained.")
0270:         error = self.score(sequence, target)
0271:         return error > self.threshold_, error
0272:
0273:     def save(self, path: Path | str = LSTM_MODEL_PATH) -> None:
0274:         if self.feature_mean_ is None or self.feature_std_ is None or self.threshold_ is
None:
0275:             raise ValueError("Detector is not trained.")
0276:         path = Path(path)
0277:         np.savez(
0278:             path,
0279:             feature_count=self.feature_count,
0280:             sequence_length=self.sequence_length,
0281:             hidden_size=self.hidden_size,
0282:             epochs=self.epochs,
0283:             learning_rate=self.learning_rate,
0284:             threshold_quantile=self.threshold_quantile,
0285:             random_state=self.random_state,
0286:             feature_mean=self.feature_mean_,
0287:             feature_std=self.feature_std_,
0288:             threshold=self.threshold_,
0289:             training_loss=np.asarray(self.training_loss_, dtype=float),
0290:             **self.model.parameters,
0291:         )
0292:
0293:     @classmethod

```

```

0294:     def load(cls, path: Path | str = LSTM_MODEL_PATH) -> "LSTMANomalyDetector":
0295:         payload = np.load(Path(path))
0296:         detector = cls(
0297:             feature_count=int(payload["feature_count"]),
0298:             sequence_length=int(payload["sequence_length"]),
0299:             hidden_size=int(payload["hidden_size"]),
0300:             epochs=int(payload["epochs"]),
0301:             learning_rate=float(payload["learning_rate"]),
0302:             threshold_quantile=float(payload["threshold_quantile"]),
0303:             random_state=int(payload["random_state"]),
0304:         )
0305:         detector.feature_mean_ = np.asarray(payload["feature_mean"], dtype=float)
0306:         detector.feature_std_ = np.asarray(payload["feature_std"], dtype=float)
0307:         detector.threshold_ = float(payload["threshold"])
0308:         detector.training_loss_ = payload["training_loss"].astype(float).tolist()
0309:         for name in detector.model.parameters:
0310:             detector.model.parameters[name] = np.asarray(payload[name], dtype=float)
0311:         return detector
0312:
0313:
0314:     def save_metadata(metadata: dict[str, float | int | str | tuple[str, ...]]) -> None:
0315:         ensure_artifact_dir()
0316:         joblib.dump(metadata, METADATA_PATH)
0317:
0318:
0319:     def load_metadata() -> dict[str, float | int | str | tuple[str, ...]]:
0320:         return joblib.load(METADATA_PATH)

```

Listing: backend.py

```

0001: from __future__ import annotations
0002:
0003: from collections import defaultdict, deque
0004: from pathlib import Path
0005:
0006: import joblib
0007: import numpy as np
0008: import pandas as pd
0009: from fastapi import FastAPI
0010: from fastapi.middleware.cors import CORSMiddleware
0011:
0012: from anomaly_detection import (
0013:     FEATURE_NAMES,
0014:     ISOLATION_FOREST_PATH,
0015:     LSTM_MODEL_PATH,
0016:     METADATA_PATH,
0017:     LSTMANomalyDetector,
0018:     load_metadata,
0019: )
0020:
0021:
0022: MAX_VISIBLE_READINGS = 60
0023: LSTM_ALERT_CONFIDENCE = 0.85
0024:
0025: app = FastAPI()
0026:
0027: app.add_middleware(
0028:     CORSMiddleware,
0029:     allow_origins=["*"],
0030:     allow_methods=["*"],
0031:     allow_headers=["*"],
0032: )
0033:
0034:
0035: def require_artifact(path: Path) -> Path:
0036:     if not path.exists():
0037:         raise RuntimeError(f"Missing model artifact: {path}. Run `python train_model.py` first.")
0038:     return path
0039:
0040:
0041: metadata = load_metadata() if METADATA_PATH.exists() else {}
0042: isolation_forest = joblib.load(require_artifact(ISOLATION_FOREST_PATH))
0043: lstm_detector = LSTMANomalyDetector.load(require_artifact(LSTM_MODEL_PATH))
0044: sequence_length = lstm_detector.sequence_length
0045:

```

```

0046: DATA: list[dict] = []
0047: DEVICE_HISTORY: dict[str, deque[np.ndarray]] = defaultdict(lambda:
    deque(maxlen=sequence_length))
0048:
0049:
0050: def build_feature_vector(item: dict) -> np.ndarray:
0051:     return np.asarray([float(item[name]) for name in FEATURE_NAMES], dtype=float)
0052:
0053:
0054: def build_feature_frame(features: np.ndarray) -> pd.DataFrame:
0055:     return pd.DataFrame([features], columns=FEATURE_NAMES)
0056:
0057:
0058: def isolation_forest_confidence(score: float) -> float:
0059:     return float(1.0 / (1.0 + np.exp(5.0 * score)))
0060:
0061:
0062: def lstm_confidence(error: float, threshold: float) -> float:
0063:     if threshold <= 0.0:
0064:         return 0.0
0065:     ratio = error / threshold
0066:     return float(max(0.0, min(0.999, 1.0 - np.exp(-ratio))))
0067:
0068:
0069: def classify_event(power: float, is_anomaly: bool) -> str:
0070:     if not is_anomaly:
0071:         return "NORMAL"
0072:     if power <= float(metadata.get("power_low_threshold", 0.5)):
0073:         return "THEFT"
0074:     if power >= float(metadata.get("power_high_threshold", 5.0)):
0075:         return "FAULT"
0076:     return "ANOMALY"
0077:
0078:
0079: def combine_confidence(
0080:     prediction: str,
0081:     iforest_is_anomaly: bool,
0082:     iforest_conf: float,
0083:     lstm_supports_alert: bool,
0084:     lstm_conf: float | None,
0085: ) -> float:
0086:     if prediction == "ANOMALY":
0087:         signals = []
0088:         if iforest_is_anomaly:
0089:             signals.append(iforest_conf)
0090:         if lstm_supports_alert:
0091:             signals.append(float(lstm_conf or 0.0))
0092:         return round(max(signals) if signals else 0.5, 4)
0093:
0094:     normal_signals = [1.0 - iforest_conf]
0095:     if lstm_conf is not None:
0096:         normal_signals.append(1.0 - lstm_conf)
0097:     return round(float(sum(normal_signals) / len(normal_signals)), 4)
0098:
0099:
0100: def analyze_item(item: dict) -> dict:
0101:     device_id = str(item.get("device_id") or "METER")
0102:     features = build_feature_vector(item)
0103:     power = float(features[2])
0104:     feature_frame = build_feature_frame(features)
0105:
0106:     iforest_score = float(isolation_forest.decision_function(feature_frame)[0])
0107:     iforest_is_anomaly = bool(isolation_forest.predict(feature_frame)[0] == -1)
0108:     iforest_conf = isolation_forest_confidence(iforest_score)
0109:
0110:     history = DEVICE_HISTORY[device_id]
0111:     lstm_ready = len(history) >= lstm_detector.sequence_length
0112:     lstm_is_anomaly: bool | None = None
0113:     lstm_error: float | None = None
0114:     lstm_conf: float | None = None
0115:
0116:     if lstm_ready:
0117:         sequence = np.asarray([entry[2] for entry in history], dtype=float)
0118:         lstm_target = np.asarray([power], dtype=float)
0119:         lstm_is_anomaly, lstm_error = lstm_detector.predict(sequence, lstm_target)
0120:         lstm_conf = lstm_confidence(lstm_error, float(metadata.get("lstm_threshold",
0.0)))
0121:

```

```

0122:     lstm_drives_alert = bool(lstm_is_anomaly) and float(lstm_conf or 0.0) >=
LSTM_ALERT_CONFIDENCE
0123: combined_anomaly = iforest_is_anomaly or lstm_drives_alert
0124: prediction = "ANOMALY" if combined_anomaly else "NORMAL"
0125: detected_event = classify_event(power, combined_anomaly)
0126: confidence = combine_confidence(
0127:     prediction=prediction,
0128:     iforest_is_anomaly=iforest_is_anomaly,
0129:     iforest_conf=iforest_conf,
0130:     lstm_supports_alert=lstm_drives_alert,
0131:     lstm_conf=lstm_conf,
0132: )
0133:
0134: anomaly_sources = []
0135: if iforest_is_anomaly:
0136:     anomaly_sources.append("Isolation Forest")
0137: if lstm_drives_alert:
0138:     anomaly_sources.append("LSTM")
0139:
0140: history.append(features)
0141:
0142: enriched_item = dict(item)
0143: enriched_item["prediction"] = prediction
0144: enriched_item["detected_event"] = detected_event
0145: enriched_item["confidence"] = confidence
0146: enriched_item["match"] = enriched_item.get("label") == detected_event
0147: enriched_item["anomaly_source"] = ", ".join(anomaly_sources) if anomaly_sources else
"None"
0148: enriched_item["algorithms"] = {
0149:     "isolation_forest": {
0150:         "is_anomaly": iforest_is_anomaly,
0151:         "score": round(iforest_score, 4),
0152:         "confidence": round(iforest_conf, 4),
0153:     },
0154:     "lstm": {
0155:         "status": "ready" if lstm_ready else "warmup",
0156:         "is_anomaly": lstm_is_anomaly,
0157:         "score": None if lstm_error is None else round(lstm_error, 4),
0158:         "confidence": None if lstm_conf is None else round(lstm_conf, 4),
0159:         "required_history": lstm_detector.sequence_length,
0160:         "history_available": len(history),
0161:     },
0162: }
0163: return enriched_item
0164:
0165:
0166: @app.post("/data")
0167: def receive(item: dict):
0168:     enriched_item = analyze_item(item)
0169:     DATA.append(enriched_item)
0170:     if len(DATA) > MAX_VISIBLE_READINGS:
0171:         del DATA[:-MAX_VISIBLE_READINGS]
0172:     return {"ok": True, "prediction": enriched_item["prediction"]}
0173:
0174:
0175: @app.get("/data")
0176: def get_data():
0177:     return DATA

```

Listing: meter_simulator.py

```

0001: from __future__ import annotations
0002:
0003: import argparse
0004: import random
0005: import time
0006: from datetime import datetime
0007:
0008: import requests
0009:
0010:
0011: DEVICE_IDS = ["METER_1", "METER_2", "METER_3"]
0012:
0013:
0014: def generate_data(device_id: str) -> dict:
0015:     voltage = random.uniform(210, 240)

```

```

0016:     current = random.uniform(5, 15)
0017:     base_power = voltage * current / 1000
0018:
0019:     hour = datetime.now().hour
0020:     if 18 <= hour <= 23:
0021:         base_power *= 1.5
0022:     elif 0 <= hour <= 6:
0023:         base_power *= 0.55
0024:
0025:     label = "NORMAL"
0026:     random_draw = random.random()
0027:     if random_draw < 0.03:
0028:         base_power = random.uniform(0.0, 0.35)
0029:         label = "THEFT"
0030:     elif random_draw < 0.06:
0031:         base_power = random.uniform(20.0, 30.0)
0032:         label = "FAULT"
0033:
0034:     return {
0035:         "device_id": device_id,
0036:         "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
0037:         "voltage": round(voltage, 2),
0038:         "current": round(current, 2),
0039:         "power": round(base_power, 2),
0040:         "label": label,
0041:     }
0042:
0043:
0044: def create_parser() -> argparse.ArgumentParser:
0045:     parser = argparse.ArgumentParser(description="Smart meter live simulator")
0046:     parser.add_argument(
0047:         "--backend-url",
0048:         default="http://localhost:8000/data",
0049:         help="Backend endpoint that receives simulated readings.",
0050:     )
0051:     parser.add_argument(
0052:         "--interval",
0053:         type=float,
0054:         default=2.0,
0055:         help="Seconds between each batch of device readings.",
0056:     )
0057:     return parser
0058:
0059:
0060: def main() -> None:
0061:     args = create_parser().parse_args()
0062:     session = requests.Session()
0063:     print("Starting live meter simulator in direct API mode...", flush=True)
0064:
0065:     while True:
0066:         for device_id in DEVICE_IDS:
0067:             payload = generate_data(device_id)
0068:             try:
0069:                 session.post(args.backend_url, json=payload, timeout=5)
0070:                 print(payload, flush=True)
0071:             except requests.RequestException as exc:
0072:                 print(f"Failed to send reading to backend: {exc}", flush=True)
0073:             time.sleep(args.interval)
0074:
0075:
0076: if __name__ == "__main__":
0077:     main()

```

Listing: flask_app.py

```

0001: from __future__ import annotations
0002:
0003: from flask import Flask, jsonify, render_template
0004: import requests
0005:
0006:
0007: BACKEND_URL = "http://localhost:8000/data"
0008:
0009: app = Flask(__name__)
0010:
0011:

```

```

0012: @app.route("/")
0013: def index():
0014:     return render_template("index.html")
0015:
0016:
0017: @app.route("/data")
0018: def data():
0019:     try:
0020:         response = requests.get(BACKEND_URL, timeout=5)
0021:         response.raise_for_status()
0022:         return jsonify(response.json())
0023:     except requests.RequestException:
0024:         return jsonify([])
0025:
0026:
0027: if __name__ == "__main__":
0028:     app.run(port=5000, debug=True)

```

Listing: run_project.py

```

0001: from __future__ import annotations
0002:
0003: import argparse
0004: import signal
0005: import socket
0006: import subprocess
0007: import sys
0008: import threading
0009: import time
0010: import traceback
0011: from datetime import datetime
0012: from pathlib import Path
0013:
0014:
0015: ROOT = Path(__file__).resolve().parent
0016: LOG_DIR = ROOT / "logs"
0017: PYTHON = sys.executable
0018: LOG_FILE = None
0019: LOG_LOCK = threading.Lock()
0020: DEBUG_MODE = False
0021:
0022:
0023: def parse_bool(value: str) -> bool:
0024:     return str(value).strip().lower() in {"1", "true", "yes", "y", "on"}
0025:
0026:
0027: def init_logging() -> Path:
0028:     global LOG_FILE
0029:     LOG_DIR.mkdir(exist_ok=True)
0030:     log_path = LOG_DIR / f"run_project_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log"
0031:     LOG_FILE = log_path.open("a", encoding="utf-8")
0032:     return log_path
0033:
0034:
0035: def close_logging() -> None:
0036:     global LOG_FILE
0037:     if LOG_FILE is not None:
0038:         LOG_FILE.close()
0039:         LOG_FILE = None
0040:
0041:
0042: def write_log_line(line: str) -> None:
0043:     with LOG_LOCK:
0044:         if LOG_FILE is not None:
0045:             print(line, file=LOG_FILE, flush=True)
0046:
0047:
0048: def print_step(message: str, level: str = "INFO") -> None:
0049:     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
0050:     line = f"[{timestamp}] [{level}] [run_project] {message}"
0051:     if level != "DEBUG":
0052:         print(line, flush=True)
0053:     if DEBUG_MODE:
0054:         write_log_line(line)
0055:
0056:

```

```

0057: def log_environment(interval: float) -> None:
0058:     print_step(f"Working directory: {ROOT}", "DEBUG")
0059:     print_step(f"Python executable: {PYTHON}", "DEBUG")
0060:     print_step(f"Simulator interval: {interval}", "DEBUG")
0061:
0062:
0063: def stream_subprocess_output(name: str, process: subprocess.Popen) -> None:
0064:     assert process.stdout is not None
0065:     for line in process.stdout:
0066:         timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
0067:         write_log_line(f"[{timestamp}] [DEBUG] [{name}] {line.rstrip()}")
0068:
0069:
0070: def run_command(name: str, command: list[str]) -> None:
0071:     print_step(f"Running command for {name}: {' '.join(command)}", "DEBUG")
0072:     kwargs = {
0073:         "cwd": ROOT,
0074:         "check": True,
0075:         "text": True,
0076:     }
0077:     if DEBUG_MODE:
0078:         kwargs["stdout"] = LOG_FILE
0079:         kwargs["stderr"] = subprocess.STDOUT
0080:     else:
0081:         kwargs["stdout"] = subprocess.DEVNULL
0082:         kwargs["stderr"] = subprocess.STDOUT
0083:     subprocess.run(command, **kwargs)
0084:
0085:
0086: def wait_for_port(host: str, port: int, timeout: float = 45.0) -> bool:
0087:     deadline = time.time() + timeout
0088:     while time.time() < deadline:
0089:         try:
0090:             with socket.create_connection((host, port), timeout=1.0):
0091:                 return True
0092:         except OSError:
0093:             time.sleep(0.5)
0094:     return False
0095:
0096:
0097: def run_training() -> None:
0098:     print_step("Training Isolation Forest and LSTM anomaly detectors...")
0099:     run_command("Model training", [PYTHON, "train_model.py"])
0100:     print_step("Model training complete.")
0101:
0102:
0103: def start_process(name: str, command: list[str]) -> subprocess.Popen:
0104:     print_step(f"Starting {name}: {' '.join(command)}")
0105:     if DEBUG_MODE:
0106:         process = subprocess.Popen(
0107:             command,
0108:             cwd=ROOT,
0109:             stdout=subprocess.PIPE,
0110:             stderr=subprocess.STDOUT,
0111:             text=True,
0112:             bufsize=1,
0113:         )
0114:         threading.Thread(
0115:             target=stream_subprocess_output,
0116:             args=(name, process),
0117:             daemon=True,
0118:         ).start()
0119:         return process
0120:
0121:     return subprocess.Popen(
0122:         command,
0123:         cwd=ROOT,
0124:         stdout=subprocess.DEVNULL,
0125:         stderr=subprocess.STDOUT,
0126:         text=True,
0127:     )
0128:
0129:
0130: def terminate_process(name: str, process: subprocess.Popen) -> None:
0131:     if process.poll() is None:
0132:         print_step(f"Stopping {name}...")
0133:         process.terminate()
0134:         try:

```

```

0135:         process.wait(timeout=5)
0136:     except subprocess.TimeoutExpired:
0137:         print_step(f"Force killing {process.pid} after timeout.", "WARNING")
0138:         process.kill()
0139:
0140:
0141: def terminate_processes(processes: list[tuple[str, subprocess.Popen]]) -> None:
0142:     for name, process in reversed(processes):
0143:         terminate_process(name, process)
0144:
0145:
0146: def main() -> None:
0147:     global DEBUG_MODE
0148:
0149:     parser = argparse.ArgumentParser(description="Run Smart Meter project services in
direct mode")
0150:     parser.add_argument(
0151:         "--debug",
0152:         default="false",
0153:         help="Set to true to save full debug logs under logs/.",
0154:     )
0155:     parser.add_argument(
0156:         "--interval",
0157:         type=float,
0158:         default=2.0,
0159:         help="Seconds between simulator batches.",
0160:     )
0161:     parser.add_argument(
0162:         "--skip-training",
0163:         default="false",
0164:         help="Set to true to reuse existing detector artifacts without retraining.",
0165:     )
0166:     args = parser.parse_args()
0167:     DEBUG_MODE = parse_bool(args.debug)
0168:     skip_training = parse_bool(args.skip_training)
0169:
0170:     if DEBUG_MODE:
0171:         log_path = init_logging()
0172:         print_step(f"Saving logs to {log_path}")
0173:         log_environment(args.interval)
0174:     else:
0175:         print_step("Running in normal mode. Use --debug true to save full logs.")
0176:
0177:     print_step("Launching the project")
0178:     if not skip_training:
0179:         run_training()
0180:     else:
0181:         print_step("Skipping training and reusing existing detector artifacts.")
0182:
0183:     processes: list[tuple[str, subprocess.Popen]] = []
0184:
0185:     def handle_signal(signum, _frame) -> None:
0186:         print_step(f"Received signal {signum}. Shutting down...")
0187:         terminate_processes(processes)
0188:         raise SystemExit(0)
0189:
0190:     signal.signal(signal.SIGINT, handle_signal)
0191:     signal.signal(signal.SIGTERM, handle_signal)
0192:
0193:     try:
0194:         backend_process = start_process(
0195:             "FastAPI backend",
0196:             [PYTHON, "-m", "uvicorn", "backend:app", "--host", "0.0.0.0", "--port",
"8000"],
0197:         )
0198:         processes.append(("FastAPI backend", backend_process))
0199:         if not wait_for_port("127.0.0.1", 8000):
0200:             raise RuntimeError("FastAPI backend did not become ready on port 8000.")
0201:
0202:         flask_process = start_process("Flask dashboard", [PYTHON, "flask_app.py"])
0203:         processes.append(("Flask dashboard", flask_process))
0204:
0205:         simulator_process = start_process(
0206:             "Meter simulator",
0207:             [PYTHON, "meter_simulator.py", "--interval", str(args.interval)],
0208:         )
0209:         processes.append(("Meter simulator", simulator_process))
0210:

```

```

0211:         print_step("Project is running.")
0212:         print_step("Dashboard: http://localhost:5000")
0213:         print_step("API: http://localhost:8000/data")
0214:         print_step("Press Ctrl+C to stop all services.")
0215:
0216:         while True:
0217:             for name, process in processes:
0218:                 code = process.poll()
0219:                 if code is not None:
0220:                     print_step(f"{name} exited unexpectedly with code {code}.", "ERROR")
0221:                     terminate_processes(processes)
0222:                     raise SystemExit(code)
0223:                 time.sleep(1)
0224:         except Exception:
0225:             print_step("Unhandled exception in launcher:", "ERROR")
0226:             for line in traceback.format_exc().rstrip().splitlines():
0227:                 print_step(line, "ERROR")
0228:             raise
0229:         finally:
0230:             terminate_processes(processes)
0231:             close_logging()
0232:
0233:
0234: if __name__ == "__main__":
0235:     main()

```

Listing: templates/index.html

```

0001: <!DOCTYPE html>
0002: <html lang="en">
0003: <head>
0004:     <meta charset="UTF-8">
0005:     <meta name="viewport" content="width=device-width, initial-scale=1.0">
0006:     <title>Smart Meter Dashboard</title>
0007:     <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
0008:     <style>
0009:         :root {
0010:             --bg: #f4flea;
0011:             --panel: #fffdf8;
0012:             --ink: #1f2937;
0013:             --muted: #6b7280;
0014:             --grid: rgba(31, 41, 55, 0.08);
0015:             --accent: #0f766e;
0016:             --danger: #c1121f;
0017:             --warning: #7c3aed;
0018:             --ok: #166534;
0019:             --anomaly: #92400e;
0020:             --border: #e5ded0;
0021:             --chart-line: #2563eb;
0022:             --normal-point: #111827;
0023:         }
0024:
0025:         * {
0026:             box-sizing: border-box;
0027:         }
0028:
0029:         body {
0030:             margin: 0;
0031:             font-family: Georgia, "Times New Roman", serif;
0032:             background:
0033:                 radial-gradient(circle at top right, rgba(15, 118, 110, 0.10),
transparent 24%),
0034:                 linear-gradient(180deg, #f9f5ee 0%, var(--bg) 100%);
0035:             color: var(--ink);
0036:         }
0037:
0038:         .page {
0039:             width: min(1200px, calc(100% - 32px));
0040:             margin: 24px auto 48px;
0041:         }
0042:
0043:         .hero {
0044:             background: linear-gradient(135deg, #fffefb, #f6efef);
0045:             border: 1px solid var(--border);
0046:             border-radius: 20px;
0047:             padding: 24px;

```

```

0048:         box-shadow: 0 16px 40px rgba(31, 41, 55, 0.08);
0049:     }
0050:
0051:     .eyebrow {
0052:         margin: 0 0 8px;
0053:         color: var(--accent);
0054:         font-size: 14px;
0055:         letter-spacing: 0.08em;
0056:         text-transform: uppercase;
0057:     }
0058:
0059:     h1 {
0060:         margin: 0;
0061:         font-size: clamp(28px, 4vw, 44px);
0062:         line-height: 1.1;
0063:     }
0064:
0065:     .subtitle {
0066:         margin: 12px 0 0;
0067:         color: var(--muted);
0068:         max-width: 760px;
0069:         font-size: 17px;
0070:     }
0071:
0072:     .status-bar {
0073:         margin-top: 18px;
0074:         display: flex;
0075:         flex-wrap: wrap;
0076:         gap: 12px;
0077:         color: var(--muted);
0078:     }
0079:
0080:     .status-pill {
0081:         padding: 8px 12px;
0082:         border-radius: 999px;
0083:         border: 1px solid var(--border);
0084:         background: rgba(255, 255, 255, 0.75);
0085:     }
0086:
0087:     .grid {
0088:         display: grid;
0089:         grid-template-columns: repeat(4, 1fr);
0090:         gap: 16px;
0091:         margin-top: 20px;
0092:     }
0093:
0094:     .card {
0095:         background: var(--panel);
0096:         border: 1px solid var(--border);
0097:         border-radius: 18px;
0098:         padding: 18px;
0099:         box-shadow: 0 10px 30px rgba(31, 41, 55, 0.05);
0100:     }
0101:
0102:     .card-label {
0103:         color: var(--muted);
0104:         font-size: 14px;
0105:         margin-bottom: 8px;
0106:     }
0107:
0108:     .card-value {
0109:         font-size: 32px;
0110:         font-weight: bold;
0111:     }
0112:
0113:     .layout {
0114:         display: grid;
0115:         grid-template-columns: 2fr 1fr;
0116:         gap: 18px;
0117:         margin-top: 20px;
0118:     }
0119:
0120:     .section-title {
0121:         margin: 0 0 10px;
0122:         font-size: 22px;
0123:     }
0124:
0125:     .section-note {

```

```

0126:         margin: 0 0 18px;
0127:         color: var(--muted);
0128:         font-size: 14px;
0129:     }
0130:
0131:     .chart-wrap {
0132:         height: 420px;
0133:     }
0134:
0135:     .chart-toolbar {
0136:         display: flex;
0137:         justify-content: space-between;
0138:         align-items: center;
0139:         gap: 12px;
0140:         margin-bottom: 14px;
0141:         flex-wrap: wrap;
0142:     }
0143:
0144:     .chart-status {
0145:         color: var(--muted);
0146:         font-size: 14px;
0147:     }
0148:
0149:     .chart-controls {
0150:         display: flex;
0151:         gap: 10px;
0152:         align-items: center;
0153:     }
0154:
0155:     .control-button {
0156:         border: 1px solid var(--border);
0157:         border-radius: 999px;
0158:         background: #fff;
0159:         color: var(--ink);
0160:         padding: 8px 14px;
0161:         font-size: 14px;
0162:         font-weight: bold;
0163:         cursor: pointer;
0164:         transition: transform 0.15s ease, box-shadow 0.15s ease;
0165:     }
0166:
0167:     .control-button:hover {
0168:         transform: translateY(-1px);
0169:         box-shadow: 0 8px 18px rgba(31, 41, 55, 0.08);
0170:     }
0171:
0172:     .control-button.paused {
0173:         background: #fff7ed;
0174:         color: var(--warning);
0175:         border-color: rgba(124, 58, 237, 0.28);
0176:     }
0177:
0178:     .events {
0179:         display: flex;
0180:         flex-direction: column;
0181:         gap: 10px;
0182:         max-height: 420px;
0183:         overflow-y: auto;
0184:     }
0185:
0186:     .event {
0187:         padding: 14px;
0188:         border: 1px solid var(--border);
0189:         border-radius: 14px;
0190:         background: #fff;
0191:     }
0192:
0193:     .event-header {
0194:         display: flex;
0195:         justify-content: space-between;
0196:         gap: 12px;
0197:         align-items: center;
0198:         margin-bottom: 8px;
0199:     }
0200:
0201:     .event-device {
0202:         font-weight: bold;
0203:     }

```

```

0204:
0205:     .event-time {
0206:         color: var(--muted);
0207:         font-size: 13px;
0208:     }
0209:
0210:     .event-meta {
0211:         display: flex;
0212:         flex-wrap: wrap;
0213:         gap: 8px;
0214:         margin: 8px 0;
0215:     }
0216:
0217:     .badge {
0218:         border-radius: 999px;
0219:         padding: 5px 10px;
0220:         font-size: 12px;
0221:         font-weight: bold;
0222:         color: white;
0223:     }
0224:
0225:     .badge-normal {
0226:         background: var(--ok);
0227:     }
0228:
0229:     .badge-theft {
0230:         background: var(--danger);
0231:     }
0232:
0233:     .badge-fault {
0234:         background: var(--warning);
0235:     }
0236:
0237:     .badge-anomaly {
0238:         background: var(--anomaly);
0239:     }
0240:
0241:     .badge-warmup {
0242:         background: #4b5563;
0243:     }
0244:
0245:     .metrics {
0246:         color: var(--muted);
0247:         font-size: 14px;
0248:         display: grid;
0249:         grid-template-columns: repeat(2, 1fr);
0250:         gap: 6px 12px;
0251:     }
0252:
0253:     .legend {
0254:         margin-top: 18px;
0255:         display: flex;
0256:         flex-wrap: wrap;
0257:         gap: 12px;
0258:         color: var(--muted);
0259:         font-size: 14px;
0260:     }
0261:
0262:     .legend span::before {
0263:         content: "";
0264:         display: inline-block;
0265:         width: 10px;
0266:         height: 10px;
0267:         border-radius: 50%;
0268:         margin-right: 8px;
0269:     }
0270:
0271:     .legend-power::before {
0272:         background: #2563eb;
0273:     }
0274:
0275:     .legend-normal::before {
0276:         background: #111827;
0277:     }
0278:
0279:     .legend-anomaly::before {
0280:         background: #92400e;
0281:     }

```

```

0282:
0283:         .legend-theft::before {
0284:             background: var(--danger);
0285:         }
0286:
0287:         .legend-fault::before {
0288:             background: var(--warning);
0289:         }
0290:
0291:         @media (max-width: 960px) {
0292:             .grid,
0293:             .layout {
0294:                 grid-template-columns: 1fr;
0295:             }
0296:
0297:             .chart-wrap {
0298:                 height: 320px;
0299:             }
0300:         }
0301:     </style>
0302: </head>
0303: <body>
0304:     <div class="page">
0305:         <section class="hero">
0306:             <p class="eyebrow">Direct Smart Meter Monitoring</p>
0307:             <h1>Isolation Forest + LSTM Dashboard</h1>
0308:             <p class="subtitle">
0309:                 The simulator sends readings directly to the backend. Isolation Forest
0310:                 scores point anomalies,
0311:                 the LSTM checks recent sequence behavior, and the app marks each reading
0312:                 as normal or anomaly.
0313:             </p>
0314:             <div class="status-bar">
0315:                 <div class="status-pill" id="live-status">Waiting for data...</div>
0316:                 <div class="status-pill" id="last-updated">Last updated: --</div>
0317:                 <div class="status-pill">Mode: Direct API only</div>
0318:             </div>
0319:         </section>
0320:
0321:         <section class="grid">
0322:             <div class="card">
0323:                 <div class="card-label">Total Visible Readings</div>
0324:                 <div class="card-value" id="total-readings">0</div>
0325:             </div>
0326:             <div class="card">
0327:                 <div class="card-label">Combined Anomalies</div>
0328:                 <div class="card-value" id="combined-anomaly-count">0</div>
0329:             </div>
0330:             <div class="card">
0331:                 <div class="card-label">Detected Theft-Like Events</div>
0332:                 <div class="card-value" id="detected-theft-count">0</div>
0333:             </div>
0334:             <div class="card">
0335:                 <div class="card-label">Detected Fault-Like Events</div>
0336:                 <div class="card-value" id="detected-fault-count">0</div>
0337:             </div>
0338:             <div class="card">
0339:                 <div class="card-label">Isolation Forest Alerts</div>
0340:                 <div class="card-value" id="iforest-alert-count">0</div>
0341:             </div>
0342:             <div class="card">
0343:                 <div class="card-label">LSTM Alerts</div>
0344:                 <div class="card-value" id="lstm-alert-count">0</div>
0345:             </div>
0346:             <div class="card">
0347:                 <div class="card-label">Event Match Count</div>
0348:                 <div class="card-value" id="match-count">0</div>
0349:             </div>
0350:             <div class="card">
0351:                 <div class="card-label">Latest Decision</div>
0352:                 <div class="card-value" id="latest-decision">--</div>
0353:             </div>
0354:         </section>
0355:
0356:         <section class="layout">
0357:             <div class="card">
0358:                 <h2 class="section-title">Power Timeline</h2>
0359:                 <p class="section-note">

```

```

0358:             Point color follows the detected event. Use the tooltips to compare
the combined anomaly decision with the
0359:             individual algorithm signals.
0360:         </p>
0361:         <div class="chart-toolbar">
0362:             <div class="chart-status" id="chart-status">Live updates are
running</div>
0363:             <div class="chart-controls">
0364:                 <button class="control-button" id="toggle-live"
type="button">Pause Live Graph</button>
0365:             </div>
0366:         </div>
0367:         <div class="chart-wrap">
0368:             <canvas id="chart"></canvas>
0369:         </div>
0370:         <div class="legend">
0371:             <span class="legend-power">Power</span>
0372:             <span class="legend-normal">Normal</span>
0373:             <span class="legend-anomaly">Generic anomaly</span>
0374:             <span class="legend-theft">Theft-like</span>
0375:             <span class="legend-fault">Fault-like</span>
0376:         </div>
0377:     </div>
0378:
0379:     <div class="card">
0380:         <h2 class="section-title">Recent Events</h2>
0381:         <p class="section-note">
0382:             Event cards show simulator labels, the combined anomaly decision,
and how Isolation Forest and LSTM voted.
0383:         </p>
0384:         <div class="events" id="events"></div>
0385:     </div>
0386: </section>
0387: </div>
0388:
0389: <script>
0390:     const ctx = document.getElementById("chart");
0391:     const eventsEl = document.getElementById("events");
0392:     const toggleLiveButton = document.getElementById("toggle-live");
0393:     const chartStatus = document.getElementById("chart-status");
0394:     let isPaused = false;
0395:     let pollHandle = null;
0396:
0397:     const chart = new Chart(ctx, {
0398:         type: "line",
0399:         data: {
0400:             labels: [],
0401:             datasets: [
0402:                 {
0403:                     label: "Power",
0404:                     data: [],
0405:                     borderColor: "#2563eb",
0406:                     backgroundColor: "rgba(37, 99, 235, 0.12)",
0407:                     fill: false,
0408:                     tension: 0.22,
0409:                     pointRadius: 4,
0410:                     pointHoverRadius: 6,
0411:                     pointBackgroundColor: [],
0412:                     pointBorderColor: [],
0413:                     borderWidth: 3
0414:                 }
0415:             ]
0416:         },
0417:         options: {
0418:             maintainAspectRatio: false,
0419:             responsive: true,
0420:             plugins: {
0421:                 legend: {
0422:                     display: false
0423:                 },
0424:                 tooltip: {
0425:                     callbacks: {
0426:                         label: function(context) {
0427:                             const row = context.raw.meta;
0428:                             const lstm = row.algorithms?.lstm || {};
0429:                             const iforest = row.algorithms?.isolation_forest || {};
0430:                             return [
0431:                                 `Meter ID: ${row.device_id || "METER"}`,

```

```

0432:         `Power: ${row.power} kW`,
0433:         `Simulator label: ${row.label}`,
0434:         `Combined decision: ${row.prediction}`,
0435:         `Detected event: ${row.detected_event}`,
0436:         `Source: ${row.anomaly_source}`,
0437:         `Confidence: ${formatConfidence(row.confidence)} `,
0438:         `Isolation Forest: ${iforest.is_anomaly ? "Alert" :
"Normal"} (${formatConfidence(iforest.confidence)})`,
0439:         `LSTM: ${formatLstmStatus(lstm)} `
0440:     ];
0441:     }
0442: }
0443: }
0444: },
0445: scales: {
0446:     x: {
0447:         grid: {
0448:             color: "rgba(31, 41, 55, 0.06)"
0449:         }
0450:     },
0451:     y: {
0452:         beginAtZero: true,
0453:         grid: {
0454:             color: "rgba(31, 41, 55, 0.08)"
0455:         },
0456:         title: {
0457:             display: true,
0458:             text: "Power"
0459:         }
0460:     }
0461: }
0462: }
0463: });
0464:
0465: function badgeClass(value) {
0466:     if (value === "THEFT") return "badge badge-theft";
0467:     if (value === "FAULT") return "badge badge-fault";
0468:     if (value === "ANOMALY") return "badge badge-anomaly";
0469:     return "badge badge-normal";
0470: }
0471:
0472: function warmupBadgeClass(status) {
0473:     return status === "warmup" ? "badge badge-warmup" : "badge badge-normal";
0474: }
0475:
0476: function formatConfidence(value) {
0477:     if (value === null || value === undefined || value === "") {
0478:         return "N/A";
0479:     }
0480:
0481:     const numericValue = Number(value);
0482:     if (Number.isNaN(numericValue)) {
0483:         return value;
0484:     }
0485:
0486:     return `${(numericValue * 100).toFixed(2)}%`;
0487: }
0488:
0489: function formatLstmStatus(lstm) {
0490:     if (!lstm || lstm.status === "warmup") {
0491:         const available = lstm?.history_available ?? 0;
0492:         const required = lstm?.required_history ?? 0;
0493:         return `Warmup (${available}/${required})`;
0494:     }
0495:     return `${lstm.is_anomaly ? "Alert" : "Normal"}
(${formatConfidence(lstm.confidence)})`;
0496: }
0497:
0498: function pointColor(reading) {
0499:     if (reading.detected_event === "THEFT") return "#c1121f";
0500:     if (reading.detected_event === "FAULT") return "#7c3aed";
0501:     if (reading.detected_event === "ANOMALY") return "#92400e";
0502:     return "#111827";
0503: }
0504:
0505: function updateSummary(data) {
0506:     const combinedAnomalyCount = data.filter(d => d.prediction ===
"ANOMALY").length;

```

```

0507:         const detectedTheftCount = data.filter(d => d.detected_event ===
0508: "THEFT").length;
0508:         const detectedFaultCount = data.filter(d => d.detected_event ===
0509: "FAULT").length;
0509:         const iforestAlertCount = data.filter(d =>
0510: d.algorithms?.isolation_forest?.is_anomaly).length;
0510:         const lstmAlertCount = data.filter(d => d.algorithms?.lstm?.is_anomaly ===
0511: true).length;
0511:         const matchCount = data.filter(d => d.match).length;
0512:         const latest = data[data.length - 1];
0513:
0514:         document.getElementById("total-readings").textContent = data.length;
0515:         document.getElementById("combined-anomaly-count").textContent =
0516: combinedAnomalyCount;
0516:         document.getElementById("detected-theft-count").textContent =
0517: detectedTheftCount;
0517:         document.getElementById("detected-fault-count").textContent =
0518: detectedFaultCount;
0518:         document.getElementById("iforest-alert-count").textContent =
0519: iforestAlertCount;
0519:         document.getElementById("lstm-alert-count").textContent = lstmAlertCount;
0520:         document.getElementById("match-count").textContent = matchCount;
0521:         document.getElementById("latest-decision").textContent = latest ?
0522: latest.detected_event : "--";
0523:
0523:         document.getElementById("live-status").textContent =
0524: data.length ? "Live stream connected" : "No readings received yet";
0525:         document.getElementById("last-updated").textContent =
0526: `Last updated: ${new Date().toLocaleTimeString()}`;
0527:     }
0528:
0529:     function updateChart(data) {
0530:         chart.data.labels = data.map(d => d.timestamp);
0531:         chart.data.datasets[0].data = data.map(d => ({
0532:             x: d.timestamp,
0533:             y: d.power,
0534:             meta: d
0535:         }));
0536:         chart.data.datasets[0].pointBackgroundColor = data.map(pointColor);
0537:         chart.data.datasets[0].pointBorderColor = data.map(pointColor);
0538:         chart.update();
0539:     }
0540:
0541:     function updateEvents(data) {
0542:         if (!data.length) {
0543:             eventsEl.innerHTML = "<div class='event'>No readings yet. Keep the
0544: simulator running.</div>";
0545:             return;
0546:         }
0547:         eventsEl.innerHTML = data.slice().reverse().slice(0, 12).map(d => `
0548: <div class="event">
0549:     <div class="event-header">
0550:         <div class="event-device">${d.device_id || "METER"}</div>
0551:         <div class="event-time">${d.timestamp}</div>
0552:     </div>
0553:     <div class="event-meta">
0554:         <span class="${badgeClass(d.label)}">Label: ${d.label}</span>
0555:         <span class="${badgeClass(d.prediction)}">Decision:
0556: ${d.prediction}</span>
0557:         <span class="${badgeClass(d.detected_event)}">Detected:
0558: ${d.detected_event}</span>
0559:         <span
0560: class="${warmupBadgeClass(d.algorithms?.lstm?.status)}">LSTM:
0561: ${formatLstmStatus(d.algorithms?.lstm)}</span>
0562:     </div>
0563:     <div class="metrics">
0564:         <div>Voltage: ${d.voltage}</div>
0565:         <div>Current: ${d.current}</div>
0566:         <div>Power: ${d.power}</div>
0567:         <div>Confidence: ${formatConfidence(d.confidence)}</div>
0568:         <div>Isolation Forest:
0569: ${d.algorithms?.isolation_forest?.is_anomaly ? "Alert" : "Normal"}</div>
0570:         <div>Source: ${d.anomaly_source}</div>
0571:     </div>
0572:     </div>
0573: `).join("");
0574:     }

```

```

0570:
0571:         async function fetchData() {
0572:             try {
0573:                 const res = await fetch("/data");
0574:                 const data = await res.json();
0575:                 updateSummary(data);
0576:                 updateChart(data);
0577:                 updateEvents(data);
0578:             } catch (error) {
0579:                 document.getElementById("live-status").textContent = "Dashboard could
not reach backend";
0580:             }
0581:         }
0582:
0583:         function startPolling() {
0584:             if (pollHandle !== null) {
0585:                 clearInterval(pollHandle);
0586:             }
0587:             pollHandle = setInterval(fetchData, 2000);
0588:         }
0589:
0590:         function updateLiveControl() {
0591:             toggleLiveButton.textContent = isPaused ? "Resume Live Graph" : "Pause Live
Graph";
0592:             toggleLiveButton.classList.toggle("paused", isPaused);
0593:             chartStatus.textContent = isPaused
? "Live updates are paused"
: "Live updates are running";
0596:         }
0597:
0598:         toggleLiveButton.addEventListener("click", () => {
0599:             isPaused = !isPaused;
0600:             if (isPaused) {
0601:                 if (pollHandle !== null) {
0602:                     clearInterval(pollHandle);
0603:                     pollHandle = null;
0604:                 }
0605:             } else {
0606:                 fetchData();
0607:                 startPolling();
0608:             }
0609:             updateLiveControl();
0610:         });
0611:
0612:         fetchData();
0613:         startPolling();
0614:         updateLiveControl();
0615:     </script>
0616: </body>
0617: </html>

```

Listing: tests/test_anomaly_detection.py

```

0001: from __future__ import annotations
0002:
0003: import shutil
0004: from pathlib import Path
0005: from unittest.mock import patch
0006:
0007: import numpy as np
0008: import pytest
0009:
0010: import anomaly_detection as anomaly
0011:
0012:
0013: TEST_ROOT = Path(__file__).resolve().parent
0014: TEST_RUNTIME_ROOT = TEST_ROOT / ".tmp_runtime"
0015:
0016:
0017: @pytest.fixture
0018: def workspace_tmp_dir(request: pytest.FixtureRequest):
0019:     TEST_RUNTIME_ROOT.mkdir(exist_ok=True)
0020:     temp_dir = TEST_RUNTIME_ROOT / request.node.name
0021:     if temp_dir.exists():
0022:         shutil.rmtree(temp_dir, ignore_errors=True)
0023:     temp_dir.mkdir(parents=True, exist_ok=True)

```

```

0024:
0025:     yield temp_dir
0026:
0027:     shutil.rmtree(temp_dir, ignore_errors=True)
0028:     if TEST_RUNTIME_ROOT.exists() and not any(TEST_RUNTIME_ROOT.iterdir()):
0029:         TEST_RUNTIME_ROOT.rmdir()
0030:
0031:
0032: def test_sigmoid_and_make_windows() -> None:
0033:     np.testing.assert_allclose(anomaly.sigmoid(np.asarray([0.0])), np.asarray([0.5]))
0034:
0035:     values = np.arange(1, 6, dtype=float).reshape(-1, 1)
0036:     sequences, targets = anomaly.make_windows(values, sequence_length=2)
0037:
0038:     expected_sequences = np.asarray(
0039:         [
0040:             [[1.0], [2.0]],
0041:             [[2.0], [3.0]],
0042:             [[3.0], [4.0]],
0043:         ]
0044:     )
0045:     expected_targets = np.asarray([[3.0], [4.0], [5.0]])
0046:
0047:     np.testing.assert_allclose(sequences, expected_sequences)
0048:     np.testing.assert_allclose(targets, expected_targets)
0049:
0050:
0051: def test_adam_optimizer_updates_parameters() -> None:
0052:     parameters = {"weight": np.asarray([1.0, -1.0], dtype=float)}
0053:     gradients = {"weight": np.asarray([0.5, -0.25], dtype=float)}
0054:     optimizer = anomaly.AdamOptimizer(parameters, learning_rate=0.1)
0055:
0056:     before = parameters["weight"].copy()
0057:     optimizer.step(parameters, gradients)
0058:
0059:     assert not np.array_equal(before, parameters["weight"])
0060:
0061:
0062: def test_simple_lstm_forward_backward_and_fit() -> None:
0063:     model = anomaly.SimpleLSTMRegressor(input_size=1, hidden_size=4, random_state=0)
0064:     sequence = np.asarray([[0.1], [0.2], [0.3]], dtype=float)
0065:     target = np.asarray([0.4], dtype=float)
0066:
0067:     prediction, cache = model.forward(sequence)
0068:     gradients = model.backward(target, prediction, cache)
0069:
0070:     assert prediction.shape == (1,)
0071:     assert len(cache) == 3
0072:     assert set(gradients) == set(model.parameters)
0073:     for name, gradient in gradients.items():
0074:         assert gradient.shape == model.parameters[name].shape
0075:         assert np.all(np.abs(gradient) <= 1.0 + 1e-9), name
0076:
0077:     sequences = np.asarray(
0078:         [
0079:             [[0.1], [0.2], [0.3]],
0080:             [[0.2], [0.3], [0.4]],
0081:             [[0.3], [0.4], [0.5]],
0082:             [[0.4], [0.5], [0.6]],
0083:         ],
0084:         dtype=float,
0085:     )
0086:     targets = np.asarray([[0.4], [0.5], [0.6], [0.7]], dtype=float)
0087:     losses = model.fit(sequences, targets, epochs=2, learning_rate=0.01)
0088:
0089:     assert len(losses) == 2
0090:     assert all(np.isfinite(loss) for loss in losses)
0091:
0092:
0093: def test_detector_requires_training_before_use() -> None:
0094:     detector = anomaly.LSTMANomalyDetector(sequence_length=3, hidden_size=4, epochs=1,
0095:         random_state=0)
0096:
0097:     with pytest.raises(ValueError, match="Detector is not trained"):
0098:         detector._scale(np.asarray([1.0]))
0099:
0100:     with pytest.raises(ValueError, match="Detector is not trained"):
0101:         detector.predict(np.asarray([[1.0], [1.1], [1.2]]), np.asarray([1.3]))

```

```

0101:
0102: def test_detector_fit_rejects_short_series() -> None:
0103:     detector = anomaly.LSTMANomalyDetector(sequence_length=4, hidden_size=4, epochs=1,
        random_state=0)
0104:
0105:     with pytest.raises(ValueError, match="Not enough samples"):
0106:         detector.fit(np.asarray([1.0, 1.1, 1.2, 1.3]))
0107:
0108:
0109: def test_detector_fit_handles_zero_std_and_predicts() -> None:
0110:     detector = anomaly.LSTMANomalyDetector(
0111:         sequence_length=3,
0112:         hidden_size=4,
0113:         epochs=1,
0114:         learning_rate=0.01,
0115:         threshold_quantile=90.0,
0116:         random_state=0,
0117:     )
0118:     constant_values = np.ones(8, dtype=float)
0119:
0120:     detector.fit(constant_values)
0121:     score = detector.score(np.asarray([[1.0], [1.0], [1.0]]), np.asarray([1.0]))
0122:     is_anomaly, error = detector.predict(np.asarray([[1.0], [1.0], [1.0]]),
        np.asarray([1.0]))
0123:
0124:     np.testing.assert_allclose(detector.feature_std_, np.asarray([1.0]))
0125:     assert isinstance(score, float)
0126:     assert isinstance(error, float)
0127:     assert isinstance(bool(is_anomaly), bool)
0128:
0129:
0130: def test_detector_save_and_load_round_trip(workspace_tmp_dir: Path) -> None:
0131:     detector = anomaly.LSTMANomalyDetector(
0132:         sequence_length=3,
0133:         hidden_size=4,
0134:         epochs=1,
0135:         learning_rate=0.01,
0136:         threshold_quantile=90.0,
0137:         random_state=0,
0138:     )
0139:     values = np.linspace(1.0, 2.0, 8)
0140:     detector.fit(values)
0141:
0142:     path = workspace_tmp_dir / "detector_round_trip.npz"
0143:
0144:     detector.save(path)
0145:     restored = anomaly.LSTMANomalyDetector.load(path)
0146:
0147:     sample_sequence = values[:3].reshape(-1, 1)
0148:     sample_target = np.asarray([values[3]], dtype=float)
0149:
0150:     assert restored.sequence_length == detector.sequence_length
0151:     assert restored.hidden_size == detector.hidden_size
0152:     np.testing.assert_allclose(restored.feature_mean_, detector.feature_mean_)
0153:     np.testing.assert_allclose(restored.feature_std_, detector.feature_std_)
0154:     assert restored.threshold_ == pytest.approx(detector.threshold_)
0155:     assert restored.predict(sample_sequence, sample_target) ==
        detector.predict(sample_sequence, sample_target)
0156:
0157:
0158: def test_metadata_round_trip_uses_artifact_directory(workspace_tmp_dir: Path) -> None:
0159:     payload = {"power_low_threshold": 0.5, "power_high_threshold": 5.0}
0160:
0161:     artifact_dir = workspace_tmp_dir / "artifacts"
0162:     metadata_path = artifact_dir / "metadata.joblib"
0163:     with patch.object(anomaly, "ARTIFACT_DIR", artifact_dir), patch.object(anomaly,
        "METADATA_PATH", metadata_path):
0164:         anomaly.save_metadata(payload)
0165:         loaded = anomaly.load_metadata()
0166:
0167:     assert loaded == payload

```

Listing: tests/test_backend.py

```

0001: from __future__ import annotations
0002:

```

```

0003: from collections import defaultdict, deque
0004:
0005: import numpy as np
0006: import pytest
0007: from fastapi.testclient import TestClient
0008:
0009: import backend
0010:
0011:
0012: class FakeIsolationForest:
0013:     def __init__(self, score: float, is_anomaly: bool) -> None:
0014:         self.score = score
0015:         self.is_anomaly = is_anomaly
0016:         self.frames: list = []
0017:
0018:     def decision_function(self, frame):
0019:         self.frames.append(frame.copy())
0020:         return np.asarray([self.score], dtype=float)
0021:
0022:     def predict(self, frame):
0023:         self.frames.append(frame.copy())
0024:         return np.asarray([-1 if self.is_anomaly else 1], dtype=int)
0025:
0026:
0027: class FakeLSTMDetector:
0028:     def __init__(self, sequence_length: int = 3, response: tuple[bool, float] = (False,
0029:         0.0)) -> None:
0030:         self.sequence_length = sequence_length
0031:         self.response = response
0032:         self.calls: list[tuple[np.ndarray, np.ndarray]] = []
0033:
0034:     def predict(self, sequence: np.ndarray, target: np.ndarray) -> tuple[bool, float]:
0035:         self.calls.append((np.asarray(sequence, dtype=float).copy(), np.asarray(target,
0036:             dtype=float).copy()))
0037:         return self.response
0038:
0039: @pytest.fixture(autouse=True)
0040: def restore_backend_state():
0041:     original_data = backend.DATA
0042:     original_history = backend.DEVICE_HISTORY
0043:     original_iforest = backend.isolation_forest
0044:     original_lstm = backend.lstm_detector
0045:     original_metadata = backend.metadata
0046:
0047:     backend.DATA = []
0048:     backend.metadata = {
0049:         "power_low_threshold": 0.5,
0050:         "power_high_threshold": 5.0,
0051:         "lstm_threshold": 1.0,
0052:     }
0053:     backend.DEVICE_HISTORY = defaultdict(lambda: deque(maxlen=3))
0054:
0055:     yield
0056:
0057:     backend.DATA = original_data
0058:     backend.DEVICE_HISTORY = original_history
0059:     backend.isolation_forest = original_iforest
0060:     backend.lstm_detector = original_lstm
0061:     backend.metadata = original_metadata
0062:
0063: @pytest.fixture
0064: def client():
0065:     with TestClient(backend.app) as test_client:
0066:         yield test_client
0067:
0068:
0069: def install_models(
0070:     *,
0071:     iforest_score: float,
0072:     iforest_is_anomaly: bool,
0073:     lstm_response: tuple[bool, float] = (False, 0.0),
0074:     sequence_length: int = 3,
0075:     metadata: dict | None = None,
0076: ) -> FakeLSTMDetector:
0077:     backend.isolation_forest = FakeIsolationForest(iforest_score, iforest_is_anomaly)
0078:     backend.lstm_detector = FakeLSTMDetector(sequence_length=sequence_length,

```

```

        response=lstm_response)
0079:     backend.metadata = {
0080:         "power_low_threshold": 0.5,
0081:         "power_high_threshold": 5.0,
0082:         "lstm_threshold": 1.0,
0083:         **(metadata or {}),
0084:     }
0085:     backend.DEVICE_HISTORY = defaultdict(lambda: deque(maxlen=sequence_length))
0086:     backend.DATA = []
0087:     return backend.lstm_detector
0088:
0089:
0090: def make_item(
0091:     *,
0092:     power: float,
0093:     voltage: float = 230.0,
0094:     current: float = 8.0,
0095:     device_id: str | None = "meter-1",
0096:     label: str | None = None,
0097: ) -> dict:
0098:     item = {
0099:         "voltage": voltage,
0100:         "current": current,
0101:         "power": power,
0102:     }
0103:     if device_id is not None:
0104:         item["device_id"] = device_id
0105:     if label is not None:
0106:         item["label"] = label
0107:     return item
0108:
0109:
0110: def prime_history(device_id: str, powers: list[float]) -> None:
0111:     history = backend.DEVICE_HISTORY[device_id]
0112:     for power in powers:
0113:         history.append(np.asarray([230.0, 8.0, power], dtype=float))
0114:
0115:
0116: def test_feature_builders_and_helper_logic() -> None:
0117:     item = make_item(power=1.8)
0118:     features = backend.build_feature_vector(item)
0119:     frame = backend.build_feature_frame(features)
0120:
0121:     np.testing.assert_allclose(features, np.asarray([230.0, 8.0, 1.8], dtype=float))
0122:     assert list(frame.columns) == ["voltage", "current", "power"]
0123:     assert backend.isolation_forest_confidence(0.0) == pytest.approx(0.5)
0124:     assert backend.lstm_confidence(5.0, 0.0) == 0.0
0125:     assert backend.lstm_confidence(3.0, 1.0) == pytest.approx(0.9502, abs=1e-4)
0126:     assert backend.classify_event(0.2, True) == "THEFT"
0127:     assert backend.classify_event(6.0, True) == "FAULT"
0128:     assert backend.classify_event(2.0, True) == "ANOMALY"
0129:     assert backend.classify_event(2.0, False) == "NORMAL"
0130:     assert (
0131:         backend.combine_confidence(
0132:             prediction="ANOMALY",
0133:             iforest_is_anomaly=False,
0134:             iforest_conf=0.1,
0135:             lstm_supports_alert=False,
0136:             lstm_conf=None,
0137:         )
0138:         == 0.5
0139:     )
0140:     assert (
0141:         backend.combine_confidence(
0142:             prediction="NORMAL",
0143:             iforest_is_anomaly=False,
0144:             iforest_conf=0.2,
0145:             lstm_supports_alert=False,
0146:             lstm_conf=0.4,
0147:         )
0148:         == 0.7
0149:     )
0150:
0151:
0152: def test_analyze_item_returns_normal_during_lstm_warmup() -> None:
0153:     install_models(iforest_score=0.6, iforest_is_anomaly=False)
0154:
0155:     result = backend.analyze_item(make_item(power=1.8, device_id=None, label="NORMAL"))

```

```

0156:
0157:     assert result["prediction"] == "NORMAL"
0158:     assert result["detected_event"] == "NORMAL"
0159:     assert result["anomaly_source"] == "None"
0160:     assert result["match"] is True
0161:     assert result["algorithms"]["lstm"]["status"] == "warmup"
0162:     assert result["algorithms"]["lstm"]["history_available"] == 1
0163:     assert "METER" in backend.DEVICE_HISTORY
0164:
0165:
0166: def test_analyze_item_flags_theft_from_isolation_forest() -> None:
0167:     install_models(iforest_score=-0.4, iforest_is_anomaly=True)
0168:
0169:     result = backend.analyze_item(make_item(power=0.2, label="THEFT"))
0170:
0171:     assert result["prediction"] == "ANOMALY"
0172:     assert result["detected_event"] == "THEFT"
0173:     assert result["anomaly_source"] == "Isolation Forest"
0174:     assert result["confidence"] ==
pytest.approx(round(backend.isolation_forest_confidence(-0.4), 4), abs=1e-4)
0175:
0176:
0177: def test_analyze_item_ignores_low_confidence_lstm_anomaly() -> None:
0178:     lstm_detector = install_models(
0179:         iforest_score=0.9,
0180:         iforest_is_anomaly=False,
0181:         lstm_response=(True, 0.2),
0182:         sequence_length=3,
0183:     )
0184:     prime_history("meter-1", [1.0, 1.1, 1.2])
0185:
0186:     result = backend.analyze_item(make_item(power=1.3, label="NORMAL"))
0187:
0188:     assert result["prediction"] == "NORMAL"
0189:     assert result["detected_event"] == "NORMAL"
0190:     assert result["anomaly_source"] == "None"
0191:     assert result["algorithms"]["lstm"]["status"] == "ready"
0192:     assert result["algorithms"]["lstm"]["is_anomaly"] is True
0193:     assert len(lstm_detector.calls) == 1
0194:     sequence, target = lstm_detector.calls[0]
0195:     np.testing.assert_allclose(sequence[:, 0], np.asarray([1.0, 1.1, 1.2]))
0196:     np.testing.assert_allclose(target, np.asarray([1.3]))
0197:
0198:
0199: def test_analyze_item_raises_alert_from_high_confidence_lstm_signal() -> None:
0200:     install_models(
0201:         iforest_score=0.8,
0202:         iforest_is_anomaly=False,
0203:         lstm_response=(True, 3.0),
0204:         sequence_length=3,
0205:     )
0206:     prime_history("meter-1", [1.0, 1.1, 1.2])
0207:
0208:     result = backend.analyze_item(make_item(power=3.0, label="ANOMALY"))
0209:
0210:     assert result["prediction"] == "ANOMALY"
0211:     assert result["detected_event"] == "ANOMALY"
0212:     assert result["anomaly_source"] == "LSTM"
0213:     assert result["match"] is True
0214:     assert result["confidence"] >= backend.LSTM_ALERT_CONFIDENCE
0215:
0216:
0217: def test_analyze_item_combines_both_models_for_fault_event() -> None:
0218:     install_models(
0219:         iforest_score=-0.2,
0220:         iforest_is_anomaly=True,
0221:         lstm_response=(True, 3.0),
0222:         sequence_length=3,
0223:     )
0224:     prime_history("meter-1", [4.8, 5.0, 5.1])
0225:
0226:     result = backend.analyze_item(make_item(power=6.2, label="FAULT"))
0227:
0228:     assert result["prediction"] == "ANOMALY"
0229:     assert result["detected_event"] == "FAULT"
0230:     assert result["anomaly_source"] == "Isolation Forest, LSTM"
0231:     expected = max(backend.isolation_forest_confidence(-0.2),
backend.lstm_confidence(3.0, 1.0))

```

```

0232:     assert result["confidence"] == pytest.approx(round(expected, 4), abs=1e-4)
0233:
0234:
0235: def test_device_history_is_kept_separate_per_meter() -> None:
0236:     lstm_detector = install_models(
0237:         iforest_score=0.6,
0238:         iforest_is_anomaly=False,
0239:         lstm_response=(True, 3.0),
0240:         sequence_length=2,
0241:     )
0242:
0243:     backend.analyze_item(make_item(power=1.0, device_id="meter-a"))
0244:     backend.analyze_item(make_item(power=1.0, device_id="meter-b"))
0245:     backend.analyze_item(make_item(power=1.1, device_id="meter-a"))
0246:     backend.analyze_item(make_item(power=1.2, device_id="meter-a"))
0247:
0248:     assert len(lstm_detector.calls) == 1
0249:     sequence, _ = lstm_detector.calls[0]
0250:     np.testing.assert_allclose(sequence[:, 0], np.asarray([1.0, 1.1]))
0251:     assert len(backend.DEVICE_HISTORY["meter-b"]) == 1
0252:
0253:
0254: def test_api_routes_store_recent_results_only(client, monkeypatch: pytest.MonkeyPatch)
-> None:
0255:     def fake_analyze_item(item: dict) -> dict:
0256:         return {
0257:             **item,
0258:             "prediction": "NORMAL",
0259:             "detected_event": "NORMAL",
0260:             "confidence": 0.75,
0261:             "match": True,
0262:             "anomaly_source": "None",
0263:             "algorithms": {},
0264:         }
0265:
0266:     monkeypatch.setattr(backend, "analyze_item", fake_analyze_item)
0267:
0268:     for index in range(backend.MAX_VISIBLE_READINGS + 5):
0269:         response = client.post("/data", json={"index": index})
0270:         assert response.status_code == 200
0271:         assert response.json() == {"ok": True, "prediction": "NORMAL"}
0272:
0273:     get_response = client.get("/data")
0274:
0275:     assert get_response.status_code == 200
0276:     assert len(get_response.json()) == backend.MAX_VISIBLE_READINGS
0277:     assert get_response.json()[0]["index"] == 5

```

Listing: tests/test_train_model.py

```

0001: from __future__ import annotations
0002:
0003: from unittest.mock import MagicMock, patch
0004:
0005: import numpy as np
0006: import pandas as pd
0007:
0008: import train_model
0009:
0010:
0011: def test_load_sample_frame_keeps_feature_columns_only() -> None:
0012:     raw_frame = pd.DataFrame(
0013:         {
0014:             "voltage": [230.0, 231.0],
0015:             "current": [8.0, 8.1],
0016:             "power": [1.8, 1.9],
0017:             "extra": [1, 2],
0018:         }
0019:     )
0020:
0021:     with patch.object(train_model.pd, "read_csv", return_value=raw_frame) as read_csv:
0022:         frame = train_model.load_sample_frame()
0023:
0024:         read_csv.assert_called_once_with("sample_data.csv")
0025:         assert list(frame.columns) == list(train_model.FEATURE_NAMES)
0026:         assert "extra" not in frame.columns

```

```

0027:
0028:
0029: def test_generate_simulator_normal_frame_has_expected_shape() -> None:
0030:     frame = train_model.generate_simulator_normal_frame(rows=48)
0031:
0032:     assert len(frame) == 48
0033:     assert list(frame.columns) == list(train_model.FEATURE_NAMES)
0034:     assert (frame["power"] >= 0.6).all()
0035:
0036:
0037: def test_load_training_frame_combines_sample_and_simulator_data() -> None:
0038:     sample_frame = pd.DataFrame(
0039:         [{"voltage": 230.0, "current": 8.0, "power": 1.8}],
0040:         columns=train_model.FEATURE_NAMES,
0041:     )
0042:     simulator_frame = pd.DataFrame(
0043:         [
0044:             {"voltage": 231.0, "current": 8.2, "power": 1.9},
0045:             {"voltage": 229.0, "current": 7.8, "power": 1.7},
0046:         ],
0047:         columns=train_model.FEATURE_NAMES,
0048:     )
0049:
0050:     with patch.object(train_model, "load_sample_frame", return_value=sample_frame),
0051:         patch.object(
0052:             train_model, "generate_simulator_normal_frame", return_value=simulator_frame
0053:         ):
0054:         combined, generated = train_model.load_training_frame()
0055:
0056:         assert len(combined) == 3
0057:         assert generated.equals(simulator_frame)
0058:
0059: def test_build_isolation_forest_returns_fitted_pipeline() -> None:
0060:     values = pd.DataFrame(
0061:         [
0062:             {"voltage": 230.0, "current": 8.0, "power": 1.8},
0063:             {"voltage": 231.0, "current": 8.1, "power": 1.9},
0064:             {"voltage": 232.0, "current": 8.2, "power": 2.0},
0065:             {"voltage": 233.0, "current": 8.3, "power": 2.1},
0066:             {"voltage": 234.0, "current": 8.4, "power": 2.2},
0067:         ],
0068:         columns=train_model.FEATURE_NAMES,
0069:     )
0070:
0071:     detector = train_model.build_isolation_forest(values)
0072:     predictions = detector.predict(values.iloc[[0]])
0073:
0074:     assert predictions.shape == (1,)
0075:
0076:
0077: def test_build_lstm_detector_uses_power_column_and_expected_config() -> None:
0078:     values = pd.DataFrame(
0079:         [
0080:             {"voltage": 230.0, "current": 8.0, "power": 1.8},
0081:             {"voltage": 231.0, "current": 8.1, "power": 1.9},
0082:             {"voltage": 232.0, "current": 8.2, "power": 2.0},
0083:         ],
0084:         columns=train_model.FEATURE_NAMES,
0085:     )
0086:     fake_detector = MagicMock()
0087:     fake_detector.fit.return_value = fake_detector
0088:
0089:     with patch.object(train_model, "LSTMAnomalyDetector", return_value=fake_detector) as
0090:         detector_cls:
0091:         result = train_model.build_lstm_detector(values)
0092:
0093:         detector_cls.assert_called_once_with(
0094:             feature_count=1,
0095:             sequence_length=8,
0096:             hidden_size=18,
0097:             epochs=60,
0098:             learning_rate=0.003,
0099:             threshold_quantile=99.0,
0100:             random_state=train_model.RANDOM_STATE,
0101:         )
0102:         fit_values = fake_detector.fit.call_args.args[0]
0103:         np.testing.assert_allclose(fit_values.flatten(),

```

```

        values["power"].to_numpy(dtype=float))
0103:     assert result is fake_detector
0104:
0105:
0106: def test_build_metadata_derives_runtime_thresholds() -> None:
0107:     values = pd.DataFrame(
0108:         [
0109:             {"voltage": 230.0, "current": 8.0, "power": 1.0},
0110:             {"voltage": 231.0, "current": 8.1, "power": 2.0},
0111:             {"voltage": 232.0, "current": 8.2, "power": 3.0},
0112:         ],
0113:         columns=train_model.FEATURE_NAMES,
0114:     )
0115:     fake_lstm = MagicMock(sequence_length=8, threshold=1.25)
0116:     metadata = train_model.build_metadata(values, fake_lstm)
0117:
0118:     assert metadata["features"] == train_model.FEATURE_NAMES
0119:     assert metadata["lstm_sequence_length"] == 8
0120:     assert metadata["lstm_threshold"] == 1.25
0121:     assert metadata["training_rows"] == 3
0122:     assert metadata["primary_runtime_mode"] == "direct"
0123:     assert metadata["power_low_threshold"] < values["power"].min()
0124:     assert metadata["power_high_threshold"] > values["power"].max()
0125:
0126:
0127:
0128: def test_main_persists_all_artifacts() -> None:
0129:     training_frame = pd.DataFrame(
0130:         [
0131:             {"voltage": 230.0, "current": 8.0, "power": 1.8},
0132:             {"voltage": 231.0, "current": 8.1, "power": 1.9},
0133:         ],
0134:         columns=train_model.FEATURE_NAMES,
0135:     )
0136:     lstm_training_frame = pd.DataFrame(
0137:         [{"voltage": 232.0, "current": 8.2, "power": 2.0}],
0138:         columns=train_model.FEATURE_NAMES,
0139:     )
0140:     fake_iforest = object()
0141:     fake_lstm = MagicMock()
0142:     fake_metadata = {"status": "ok"}
0143:
0144:     with patch.object(train_model, "ensure_artifact_dir") as ensure_artifact_dir,
0145:         patch.object(
0146:             train_model, "load_training_frame", return_value=(training_frame,
0147:                 lstm_training_frame)
0148:         ) as load_training_frame, patch.object(
0149:             train_model, "build_isolation_forest", return_value=fake_iforest
0150:         ) as build_isolation_forest, patch.object(
0151:             train_model, "build_lstm_detector", return_value=fake_lstm
0152:         ) as build_lstm_detector, patch.object(
0153:             train_model, "build_metadata", return_value=fake_metadata
0154:         ) as build_metadata, patch.object(
0155:             train_model.joblib, "dump"
0156:         ) as dump_model, patch.object(
0157:             train_model, "save_metadata"
0158:         ) as save_metadata, patch("builtins.print"):
0159:         train_model.main()
0160:
0161:     ensure_artifact_dir.assert_called_once_with()
0162:     load_training_frame.assert_called_once_with()
0163:     build_isolation_forest.assert_called_once_with(training_frame)
0164:     build_lstm_detector.assert_called_once_with(lstm_training_frame)
0165:     build_metadata.assert_called_once_with(training_frame, fake_lstm)
0166:     dump_model.assert_called_once_with(fake_iforest, train_model.ISOLATION_FOREST_PATH)
0167:     fake_lstm.save.assert_called_once_with(train_model.LSTM_MODEL_PATH)
0168:     save_metadata.assert_called_once_with(fake_metadata)

```